



D0 · DELIVERABLE

Submission Index

HireSense: deliverables, outcomes and rubric mapping for Case Study 31.

Start here. Six documents answer the six required deliverables, backed by a working prototype whose data, tests and code generate every number in them.

Contents

1	Deliverables required by the case study	2	Outcomes the case study asks for
3	Mapping to the course rubric	4	The working prototype

AUTHOR	COURSE	DOCUMENT
Anshuman Atrey	Software Engineering & Project Management	D0 · v1.0
Roll 150096724029	B.Tech CSE · Batch 2024–28 · Semester V	1 October 2026 · ITM Skills University

1 Deliverables required by the case study

CASE-STUDY DELIVERABLE	DOCUMENT	WHAT IT CONTAINS
White-Box Test Report: CFG, cyclomatic complexity, independent paths, coverage evidence	D1	CFG (11 nodes, 14 edges), $V(G) = 5$ by three methods, 8/8 paths tested, boundary/condition/metamorphic tests, the silent-parse-failure test, coverage, 11 bug reports
SRS: explainability, configurability, logging and fairness requirements with thresholds	D2	IEEE 830 structure, stakeholders, 22 measurable functional requirements, NFRs, use cases, traceability matrix to code and tests, feasibility
Design Pack: class diagram, activity diagram, module cohesion justification	D3	Architecture, cohesion/coupling with a measured dependency graph, class, activity, sequence, state, deployment and ER diagrams, UI screens
Audit Specification: metrics, frequency, thresholds, breach response procedure	D4	Selection rate, impact ratio, z-test, 9 acceptance tests, rolling windows, severity ladder, RACI procedure, live results
Project Plan: incremental WBS, Gantt, risk register including reputational risk	D5	Charter, 7 strangler increments, WBS, work package WP-P, COCOMO, PERT/CPM, Gantt, sprints, backlog, burndown, 12-risk register
Maintenance Process: recurring audit cycle as preventive maintenance	D6	Maintenance types, drift sources, monthly cycle and calendar, change and release gates, KPIs, audit history, maintenance log

2 Outcomes the case study asks for

EXPECTED OUTCOME	EVIDENCE
A cyclomatic complexity value with the complete independent path set and tests	D1 §3–4: $V(G) = 5$; basis paths P1–P5 plus P6–P8; every path has a passing test
A test that specifically catches silent parse failure	D1 §6: scanned résumé → legacy rejects silently; HireSense routes to review (and OCR recovers it)
Fairness expressed as a number with a defined consequence when breached	D4 §4–7: $IR \geq 0.80$ for groups with $n \geq 30$; a significant breach pauses automated rejection
A modular design where changing a ranking rule does not touch parsing	D3 §2: ranking and scoring import nothing from parsing (measured); criteria are versioned data
A recurring audit process rather than a one-off report	D4 §5 + D6 §3: monthly rolling audits, triggered audits, governance review, KPIs

3 Mapping to the course rubric

RUBRIC ITEM (MARKS)	WHERE
Problem analysis & requirement gathering (2)	D2 §1–3
SRS & documentation quality (3)	D2 (IEEE 830), D0–D6
UML design & architecture (3)	D3: use-case (D2 §6), class, activity, sequence, state, deployment, ER
Use of software-engineering tools (3)	Bun test + coverage, JUnit, Mermaid UML, SQLite, Git, Next.js/shadcn, Ollama, PP-OCR, headless-browser doc build
Project planning: WBS, Gantt, agile artefacts (3)	D5 §3–7
Testing artefacts: test cases, bug reports (2)	D1 §4–9

RUBRIC ITEM (MARKS)	WHERE
GitHub repository & version control (2)	hiresense/ monorepo (D5 §9)
Presentation & viva (2)	Live prototype demo

4 The working prototype

APPLICATIONS SCREENED

2,000

synthetic, 6 roles, 12 months

PARSE SUCCESS

99.0%

legacy 71.0%

SILENT FAILURES

580 → 6

legacy → HireSense

AUTOMATED TESTS

82 ✓

165 assertions, 0 failures

Every number in D0–D6 is generated from the running prototype by docs/build.ts : the seeded database, a fresh test run with coverage, and the code itself. The documents cannot drift from the system.

```
cd hiresense
bun run setup      # install + seed 2,000 synthetic résumés (PDFs, parses, decisions, audits)
bun run dev        # API http://localhost:8787/api · web http://localhost:3000
bun run test       # unit + integration tests with coverage
cd docs && bun run build  # regenerate these PDFs from live data
```

Stack: Next.js 16 · shadcn/ui · Bun + Hono · bun:sqlite · pdf.js · PP-OCRv6 tiny (ONNX) · optional Qwen3 1.7B via Ollama. All candidate data is synthetic. [Case-study figures used as given: routine, 0.6, 29%, 80%](#)



D1 · DELIVERABLE

White-Box Test Report

Control-flow graph, cyclomatic complexity, independent paths, the silent-parse-failure test and coverage evidence.

Structural testing of the specified screening routine and of the parser defect the case study identifies. Every number is produced by the test run and the corpus evaluation used to build this document.

Contents

1	Scope and unit under test	2	Control-flow graph
3	Cyclomatic complexity	4	Independent paths and test cases
5	Boundary values, equivalence classes and metamorphic tests	6	The silent parse failure test
7	Coverage evidence	8	Defects found during testing (bug reports)
9	Test inventory		

AUTHOR	COURSE	DOCUMENT
Anshuman Atrey	Software Engineering & Project Management	D1 · v1.0
Roll 150096724029	B.Tech CSE · Batch 2024-28 · Semester V	1 October 2026 · ITM Skills University

1 Scope and unit under test

This report applies white-box (structural) testing to the screening routine that the case study specifies, and to the defect the case study names as the real one: résumé parsing that **fails silently** on 29% of documents. The routine is implemented as one pure function, so its control flow can be read straight from the code.

Specification (case study, verbatim)

```
if mandatory skills missing, reject;
else score experience band;
  if score ≥ threshold and location matches, shortlist;
  else if score ≥ threshold and remote allowed, shortlist;
  else waitlist;
if parse confidence < 0.6, route to manual review.
```

Implementation under test

`api/src/ranking/routine.ts` → `screen(parsed, criteria)`. The function is pure: it has no I/O and no clock. Each statement carries the CFG node id it implements, and every call returns a **trace** of the nodes it visited. That trace is stored with the decision in the append-only log.

Test framework: `bun test` (Bun 1.3, Jest-compatible), with coverage from `bun test --coverage`. Results come from the run used to build this document.

```
export function screen(p: ParsedResume, c: Criteria): Decision {
  const missingSkills = c.mandatorySkills.filter((s) => !p.skills.includes(s)); // N1
  if (missingSkills.length > 0) { outcome = "reject" } // N2
  else {
    ({ score, band } = bandScore(p.yearsExperience, c.bands)); // N3
    const meets = score >= c.threshold;
    const locMatch = !!p.location && p.location.toLowerCase() === c.location.toLowerCase();
    if (meets && locMatch) outcome = "shortlist"; // N4 → N5
    else if (meets && c.remoteAllowed) outcome = "shortlist"; // N6 → N7
    else outcome = "waitlist"; // N8
  }
  if (p.confidence < CONFIDENCE_GATE) { // N9 (0.6)
    tentativeOutcome = outcome; outcome = "manual_review"; // N10
  }
  return { outcome, tentativeOutcome, score, band, reasonCode, reason, trace }; // N11
}
```

TESTS RUN

82

165 assertions

FAILURES

0

0 skipped · 2.5 s

ROUTINE.TS COVERAGE

100%

lines · 100% functions

CFG PATHS COVERED

8 / 8

all complete paths, 5 =
basis set

2 Control-flow graph

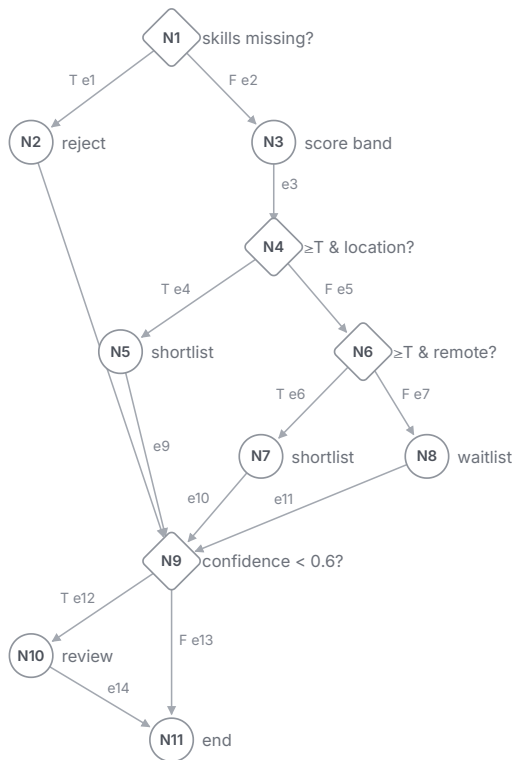


Figure 1. CFG of `screen()`. Diamonds are predicate (decision) nodes. Edges e1–e14 are labelled T/F where they leave a decision.

Table 1. Nodes and the statements they represent

NODE	STATEMENT	TYPE
N1	if mandatory skills missing	decision
N2	reject	process
N3	else score experience band	process
N4	if score $\geq$ threshold and location matches	decision
N5	shortlist	process
N6	else if score $\geq$ threshold and remote allowed	decision
N7	shortlist	process
N8	else waitlist	process
N9	if parse confidence < 0.6	decision
N10	route to manual review	process
N11	end (decision logged)	process

Table 2. Edges

EDGE	FROM $\rightarrow$ TO	CONDITION
e1	N1 $\rightarrow$ N2	true
e2	N1 $\rightarrow$ N3	false
e3	N3 $\rightarrow$ N4	—
e4	N4 $\rightarrow$ N5	true
e5	N4 $\rightarrow$ N6	false
e6	N6 $\rightarrow$ N7	true
e7	N6 $\rightarrow$ N8	false
e8	N2 $\rightarrow$ N9	—
e9	N5 $\rightarrow$ N9	—
e10	N7 $\rightarrow$ N9	—
e11	N8 $\rightarrow$ N9	—
e12	N9 $\rightarrow$ N10	true
e13	N9 $\rightarrow$ N11	false
e14	N10 $\rightarrow$ N11	—

Why N9 comes after every branch. The case study lists the confidence rule last, so it applies to every outcome, including a rejection. This turns the silent failure into a *withheld* decision. An unreadable résumé looks like "no skills" at N1, and the old system rejected it. Here N9 overrides that rejection, sends the application to a person, and keeps the automated outcome as "tentative" for the reviewer.

3 Cyclomatic complexity

M McCabe's cyclomatic complexity $V(G)$ is the number of linearly independent paths through the graph, so it is the minimum number of tests that exercises every edge. It is computed three independent ways:

METHOD	FORMULA	VALUES	V(G)
Edges and nodes	$V(G) = E - N + 2P$	$E = 14, N = 11, P = 1$ connected component	5
Predicate nodes	$V(G) = \pi + 1$	$\pi = 4$ (N1, N4, N6, N9)	5
Regions	$V(G) = \text{enclosed regions} + 1$	4 enclosed regions of the planar graph + 1 outer region	5

Condition-level view (compound predicates)

N4 and N6 are compound conditions ($\text{score} \geq T \wedge \text{location}, \text{score} \geq T \wedge \text{remote}$). Splitting them into atomic conditions gives six predicates: N1, N4a $\text{score} \geq T$, N4b location matches, N6a $\text{score} \geq T$, N6b remote allowed and N9. That makes $V(G) = 7$ at condition level. One combination is **infeasible**: N4a = false with N6a = true is impossible, because both test the same expression. The implementation therefore evaluates $\text{meets} = \text{score} \geq \text{threshold}$ once and reuses it. The condition-coverage tests in §5 exercise every feasible value of every atomic condition.

Result: $V(G) = 5$. At least 5 test cases are needed for edge (branch) coverage. The routine has 8 complete entry-to-exit paths: 4 outcomes before N9, times 2 for N9. The test suite covers all 8.

4 Independent paths and test cases

Basis set P1–P5. Each path adds at least one edge not used by the previous ones, so together they cover all 14 edges. P6–P8 complete the path set.

PATH	NODE SEQUENCE	MEANING	TEST (FILE:LINE)	RESULT
P1	N1 → N2 → N9 → N11	Mandatory skill missing, confident parse → reject	routine.test.ts:10	Pass
P2	N1 → N3 → N4 → N5 → N9 → N11	Score $\geq$ threshold, in job city → shortlist	routine.test.ts:18	Pass
P3	N1 → N3 → N4 → N6 → N7 → N9 → N11	Score $\geq$ threshold, elsewhere, remote allowed → shortlist	routine.test.ts:25	Pass
P4	N1 → N3 → N4 → N6 → N8 → N9 → N11	Below threshold → waitlist	routine.test.ts:32	Pass
P5	N1 → N2 → N9 → N10 → N11	Parse failed (no skills read, confidence 0) → manual review, not reject	routine.test.ts:39	Pass
P6 extra	N1 → N3 → N4 → N5 → N9 → N10 → N11	Would shortlist, but confidence < 0.6 → review	routine.test.ts:56	Pass
P7 extra	N1 → N3 → N4 → N6 → N7 → N9 → N10 → N11	Would shortlist (remote), confidence < 0.6 → review	routine.test.ts:62	Pass
P8 extra	N1 → N3 → N4 → N6 → N8 → N9 → N10 → N11	Would waitlist, confidence < 0.6 → review	routine.test.ts:68	Pass

Test data for the basis paths

Criteria fixture: mandatory skills `react, typescript`; bands 0–2 → 20, 2–4 → 55, 4–7 → 85, 7+ → 90; threshold 80; job city Bengaluru; remote allowed, unless a row says otherwise.

TC	PATH	SKILLS	YEARS	CITY	CONFIDENCE	CRITERIA CHANGE	EXPECTED OUTCOME / REASON CODE
TC-01	P1	react	5	Bengaluru	0.90	—	reject · MISSING_MANDATORY_SKILLS (typescript)

TC	PATH	SKILLS	YEARS	CITY	CONFIDENCE	CRITERIA CHANGE	EXPECTED OUTCOME / REASON CODE
TC-02	P2	react, typescript, css	5	Bengaluru	0.90	—	shortlist · SHORTLIST_LOCATION
TC-03	P3	react, typescript, css	5	Pune	0.90	—	shortlist · SHORTLIST_REMOTE
TC-04	P4	react, typescript, css	3	Bengaluru	0.90	—	waitlist · WAITLIST_BELOW_THRESHOLD (score 55)
TC-05	P5	— (none read)	unknown	unknown	0.00	—	manual_review · LOW_PARSE_CONFIDENCE, tentative reject
TC-06	P6	react, typescript, css	5	Bengaluru	0.45	—	manual_review, tentative shortlist
TC-07	P7	react, typescript, css	5	Pune	0.40	—	manual_review, tentative shortlist
TC-08	P8	react, typescript, css	3	Bengaluru	0.40	—	manual_review, tentative waitlist
TC-09	N6 F (location)	react, typescript, css	5	Pune	0.90	remote not allowed	waitlist · WAITLIST_LOCATION

5 Boundary values, equivalence classes and metamorphic tests

INPUT	EQUIVALENCE CLASSES	BOUNDARY TESTS (VALUE → EXPECTED)	RESULT
score vs threshold	below · at · above	score 85, threshold 85 → shortlist (≥ is inclusive); threshold 86 → waitlist	Pass
parse confidence	< 0.6 · ≥ 0.6	0.60 → decided automatically; 0.59 → manual review	Pass
years vs band edge	[2,4) · [4,7)	4.00 → score 85; 3.99 → score 55	Pass
years unknown	null	null → score 0 (never an error)	Pass
mandatory list	empty · non-empty	no mandatory skills → nothing can be missing	Pass
location text	case variants	"bengaluru" = "Bengaluru"	Pass
location unknown	null	null never matches the job city	Pass

Condition coverage (atomic conditions of N4 and N6)

MEETS (SCORE ≥ T)	LOCATION MATCHES	REMOTE ALLOWED	OUTCOME	COVERED BY
true	true	—	shortlist (N5)	TC-02
true	false	true	shortlist (N7)	TC-03
true	false	false	waitlist, location	TC-09
false	— (short-circuit)	— (short-circuit)	waitlist, below threshold	TC-04
false at N4, true at N6			infeasible: same expression	—

Metamorphic tests: attributes outside the criteria cannot change a decision

The case study's internal review found the old ranking tracked **university tier**. The relation tested here is: changing only the university (IIT Bombay ↔ Anna University ↔ none), or only the name and email, must leave the outcome and the score unchanged. Pass Pass

6 The silent parse failure test

The case study names the current system's real defect: *"résumé parsing works on 71% of documents and fails silently on the rest."* A silent failure is worse than a crash. The parser returns **empty skills** with no error, and the routine then rejects a possibly qualified candidate with a reason that looks legitimate: "missing mandatory skills". The tests below catch exactly that, on real PDFs produced by the seed templates.

TEST	INPUT	ASSERTION	RESULT
Legacy rejects silently	scanned résumé (glyph outlines, no text layer) of a candidate who has every skill	legacy parse → skills = [] and no error; legacy outcome = reject (reproduces the defect)	Pass
v2 detects it	same PDF	v2 textChars < 200 → confidence = 0, issue "no usable text layer"; routine → manual_review , tentative reject	Pass
Unreadable section	experience written in seasons ("Summer 2023 – Spring 2025")	years = null but the section has content → confidence capped at 0.50 → manual review	Pass
OCR recovery	same scanned PDF, OCR enabled (PP-OCrv6)	text recovered, skills/years/city correct, confidence ≤ OCR word confidence → decided automatically	Pass

System-level evidence on the 2,000-résumé corpus

LEGACY CORRECT 71.0% case-study figure: 71%	LEGACY SILENT FAILURES 580 every failure is silent	HIRESENSE CORRECT 99.0% 14 of 20 failures routed to review	HIRESENSE SILENT 6 0.30% of résumés
--	---	---	--

A parse is *correct* only when all three decision fields match ground truth: skills (F1 ≥ 0.8), years (±1) and city. The corpus is synthetic and built so that 29% of résumés use layouts that break the legacy parser.

7 Coverage evidence

Line and function coverage from `bun test --coverage`, for the modules that make decisions. Bun does not report branch coverage, so edge (branch) coverage of the routine comes from the path tests in §4 (all 14 edges). It is confirmed independently from production traces in the table below.

MODULE	% FUNCTIONS	% LINES	UNCOVERED LINES
src/audit/fairness.ts	100.0	100.0	–
src/audit/service.ts	100.0	100.0	–
src/db.ts	0.0	60.0	119–122
src/legacy/legacy.ts	100.0	97.4	–
src/parsing/dates.ts	100.0	100.0	–
src/parsing/llm.ts	85.7	93.3	47–53
src/parsing/ocr.ts	100.0	95.0	21–22
src/parsing/parser.ts	100.0	100.0	–

MODULE	% FUNCTIONS	% LINES	UNCOVERED LINES
src/parsing/pdf.ts	100.0	100.0	–
src/parsing/taxonomy.ts	100.0	100.0	–
src/ranking/order.ts	100.0	95.2	–
src/ranking/routine.ts	100.0	100.0	–
src/scoring/experience.ts	100.0	100.0	–
src/screening.ts	100.0	100.0	–

The remaining uncovered lines are I/O and environment wrappers: the database bootstrap, the "is the local model running" probe, and the OCR-unavailable branch. The HTTP layer (`src/index.ts`) is exercised by API smoke tests and is not part of the unit run.

Table 3. CFG paths actually taken by the 2,170 decisions in the append-only log

PATH	NODE SEQUENCE	DECISIONS
P1	N1 → N2 → N9 → N11	673
P4	N1 → N3 → N4 → N6 → N8 → N9 → N11	604
P2	N1 → N3 → N4 → N5 → N9 → N11	550
P3	N1 → N3 → N4 → N6 → N7 → N9 → N11	313
P8	N1 → N3 → N4 → N6 → N8 → N9 → N10 → N11	17
P5	N1 → N2 → N9 → N10 → N11	13

Edges exercised in production: **14 / 14**. P6 and P7 do not occur naturally in the corpus: a confident parse rarely sits next to a shortlist-level score. That is why they are covered by targeted unit tests.

8 Defects found during testing (bug reports)

Every defect below was found by a test, by the evaluation against ground truth, or by observing the running system. Each one was fixed and is now guarded by a regression test.

ID	SEVERITY	FOUND BY	DEFECT AND ROOT CAUSE	FIX / REGRESSION TEST
BUG-01	High	Corpus evaluation	Letter-spaced headings were extracted as "S K I L L S" and not recognised. Both parsers failed 721 standard résumés, and education years were counted as experience.	Collapse single-letter runs before heading match · <i>letter-spaced headings</i>
BUG-02	Med	Corpus evaluation	City detection returned the first city in list order rather than in text order (95 wrong cities when a university name contained a city).	Earliest occurrence wins · parser tests
BUG-03	High	Corpus evaluation	Two-column layouts whose sidebar held < 15% of the text were not detected, so 24 résumés merged their columns.	Balance floor 8% + side-by-side baseline check · <i>two-column</i>
BUG-04	High	Path P5 review	Season dates ("Summer 2023") gave years = null yet confidence 0.70. A new silent failure .	Anomaly cap 0.50 when a section has content but yields nothing · <i>unreadable season dates</i>
BUG-05	Low	AI assist review	"Node.js" also matched JavaScript through the "JS" alias.	Lookbehind treats "." as part of a word · <i>Node.js does not also match</i>
BUG-06	High	OCR benchmark	OCR emits U+2212 "-" and drops spaces ("Sep2021-Jul2023"), so 37 of 160 scanned résumés lost a job.	Dash set and month/year spacing widened · date-range tests
BUG-07	Med	Test writing	AI assist kept a hallucinated city whenever the résumé mentioned any other city.	City must be the one named in the text · <i>drops hallucinated</i>

ID	SEVERITY	FOUND BY	DEFECT AND ROOT CAUSE	FIX / REGRESSION TEST
BUG-08	Med	Manual testing	AI assist turned a headline plus an education year into a fake job (employer "N/A").	Employer must appear in the résumé · <i>a headline or education line is not a job</i>
BUG-09	Med	Manual testing	The local model took longer than Bun's 10 s idle timeout and the response was cut off.	idleTimeout 120 s
BUG-10	Med	User report	PDF viewer re-rendered endlessly: the scrollbar toggled the width between 444 and 429 px.	scrollbar-gutter: stable + ignore scrollbar-sized resizes
BUG-11	Med	Report writing	Fairness reported a vacuous ratio of 1.0 when only one group had $n \geq 30$.	Ratio = none + "insufficient sample" · <i>only one comparable group</i>

9 Test inventory

All 82 automated tests from the run used for this report (JUnit output of `bun test`).

FILE	SUITE	TEST	RESULT
routine.test.ts	basis paths (V(G) = 5)	P1 N1→N2→N9 → N11: mandatory skill missing, confident parse → reject	Pass
routine.test.ts	basis paths (V(G) = 5)	P2 N1→N3→N4→N5→N9 → N11: score $\geq$ threshold and location matches → shortlist	Pass
routine.test.ts	basis paths (V(G) = 5)	P3 N1→N3→N4→N6→N7→N9 → N11: score $\geq$ threshold, elsewhere, remote allowed → shortlist	Pass
routine.test.ts	basis paths (V(G) = 5)	P4 N1→N3→N4→N6→N8→N9 → N11: below threshold → waitlist	Pass
routine.test.ts	basis paths (V(G) = 5)	P5 ...→N9→N10 → N11: parse confidence < 0.6 → manual review (the silent-failure path)	Pass
routine.test.ts	remaining branch outcomes	score $\geq$ threshold, elsewhere, remote NOT allowed → waitlist for location	Pass
routine.test.ts	remaining branch outcomes	low confidence overrides a shortlist too (automated outcome is only tentative)	Pass
routine.test.ts	remaining branch outcomes	remote shortlist withheld by low confidence: N1→N3→N4→N6→N7→N9→N10	Pass
routine.test.ts	remaining branch outcomes	waitlist withheld by low confidence: N1→N3→N4→N6→N8→N9→N10	Pass
routine.test.ts	remaining branch outcomes	unknown location never matches the job city	Pass
routine.test.ts	boundary values	score exactly at threshold is shortlisted ($\geq$, not $>$)	Pass
routine.test.ts	boundary values	score one below threshold is waitlisted	Pass
routine.test.ts	boundary values	confidence exactly 0.6 is NOT routed to review (< 0.6)	Pass
routine.test.ts	boundary values	confidence 0.59 is routed to review	Pass
routine.test.ts	boundary values	band edges: 4.0 yrs falls in [4,7), 3.99 falls in [2,4)	Pass
routine.test.ts	boundary values	unparsed experience scores 0	Pass
routine.test.ts	boundary values	no mandatory skills configured → nothing can be missing	Pass
routine.test.ts	boundary values	location match is case-insensitive	Pass
routine.test.ts	metamorphic: attributes outside the criteria cannot change a decision	university = IIT Bombay	Pass
routine.test.ts	metamorphic: attributes outside the criteria cannot change a decision	university = Anna University	Pass

FILE	SUITE	TEST	RESULT
routine.test.ts	metamorphic: attributes outside the criteria cannot change a decision	university = null	Pass
routine.test.ts	metamorphic: attributes outside the criteria cannot change a decision	name and email do not affect the decision	Pass
routine.test.ts		every decision carries a human-readable reason and a trace ending at the confidence gate	Pass
audit.test.ts	4/5ths impact-ratio rule	ratio exactly 0.80 passes	Pass
audit.test.ts	4/5ths impact-ratio rule	ratio 0.78 breaches	Pass
audit.test.ts	4/5ths impact-ratio rule	groups under 30 are reported but cannot breach	Pass
audit.test.ts	4/5ths impact-ratio rule	'Prefer not to say' is never compared	Pass
audit.test.ts	4/5ths impact-ratio rule	applications still in manual review are excluded from rates	Pass
audit.test.ts	4/5ths impact-ratio rule	a small group cannot set the benchmark rate	Pass
audit.test.ts	4/5ths impact-ratio rule	only one comparable group → no ratio (not a vacuous 1.0)	Pass
audit.test.ts	4/5ths impact-ratio rule	threshold constant is the case-study value	Pass
audit.test.ts	breach severity	no breach → pass	Pass
audit.test.ts	breach severity	breach that could be noise (n = 40, ratio 0.75) → warning	Pass
audit.test.ts	breach severity	significant breach (n = 300, ratio 0.70) → critical	Pass
audit.test.ts	rank order is explicit and explainable	same outcome and score → earlier application ranks higher	Pass
audit.test.ts	rank order is explicit and explainable	higher score wins within the same outcome	Pass
audit.test.ts	rank order is explicit and explainable	outcome dominates score	Pass
audit.test.ts	decision log is append-only	UPDATE is refused	Pass
audit.test.ts	decision log is append-only	DELETE is refused	Pass
parser.test.ts	standard résumé	v2 extracts every decision field with full confidence	Pass
parser.test.ts	standard résumé	legacy parser also handles the standard layout	Pass
parser.test.ts	silent parse failure — the current system's real defect	scanned/outlined PDF: legacy silently REJECTS a qualified candidate	Pass
parser.test.ts	silent parse failure — the current system's real defect	scanned/outlined PDF: v2 detects it (confidence 0) and routes to manual review	Pass
parser.test.ts	silent parse failure — the current system's real defect	unreadable season dates cap confidence below the gate instead of scoring 0 years silently	Pass
parser.test.ts	layouts the legacy parser gets wrong	two-column: v2 reads columns separately	Pass
parser.test.ts	layouts the legacy parser gets wrong	creative headings: v2 knows 'Toolbox' / 'Where I've Worked'; legacy does not	Pass
parser.test.ts	date ranges	Jan 2019 – Jan 2022 → 3 yrs	Pass
parser.test.ts	date ranges	January 2020 - Present → 6 yrs	Pass
parser.test.ts	date ranges	03/2019 – 03/2021 → 2 yrs	Pass
parser.test.ts	date ranges	2019/03 → now → 6.8 yrs	Pass
parser.test.ts	date ranges	'18 – '22 → 4 yrs	Pass

FILE	SUITE	TEST	RESULT
parser.test.ts	date ranges	Since Jan 2024 → 2 yrs	Pass
parser.test.ts	date ranges	03.2017 - 03.2020 → 3 yrs	Pass
parser.test.ts	date ranges	2016 – 2019 → 3 yrs	Pass
parser.test.ts	date ranges	Mar 2019 – Mar 2022 → 3 yrs	Pass
parser.test.ts	date ranges	Jul2022-Jan2024 → 1.5 yrs	Pass
parser.test.ts	date ranges	overlapping jobs are not double counted	Pass
parser.test.ts	date ranges	future and inverted ranges are ignored	Pass
parser.test.ts		letter-spaced headings ('S K I L L S') are still recognised	Pass
parser.test.ts	OCR fallback (scanned PDFs)	recovers the text of a scanned résumé and passes the gate	Pass
parser.test.ts	half-credit fallbacks (fields found outside their section)	skills without a skills heading count at half weight	Pass
parser.test.ts	half-credit fallbacks (fields found outside their section)	dates without an experience heading count at half weight	Pass
parser.test.ts		parseResume parses a rendered PDF end to end	Pass
parser.test.ts		legacy score rewards university tier (the bias the internal review found)	Pass
audit-service.test.ts	runAudit	fair outcomes → pass, automated rejection stays on	Pass
audit-service.test.ts	runAudit	significant breach → critical, auto-reject paused, later rejections routed to review	Pass
audit-service.test.ts	runAudit	small-sample breach → warning, not paused	Pass
audit-service.test.ts	runAudit	a passing re-audit resumes automated rejection	Pass
audit-service.test.ts	runAudit	no comparable groups → reported as insufficient sample	Pass
llm.test.ts	toMonths	2023-04	Pass
llm.test.ts	toMonths	Apr 2023	Pass
llm.test.ts	toMonths	April 2023	Pass
llm.test.ts	toMonths	Spring 2023	Pass
llm.test.ts	toMonths	Winter 2020	Pass
llm.test.ts	toMonths	2019	Pass
llm.test.ts	toMonths	present	Pass
llm.test.ts	toMonths	today	Pass
llm.test.ts	toMonths	unreadable wording → null	Pass
llm.test.ts	grounding	keeps grounded values and converts season dates	Pass
llm.test.ts	grounding	a headline or education line is not a job (employer must appear in the résumé)	Pass
llm.test.ts	grounding	drops hallucinated skills, cities and positions	Pass
llm.test.ts		'Node.js' does not also match JavaScript's 'JS' alias	Pass



D2 · DELIVERABLE

Software Requirements Specification

Explainability, configurable criteria, audit logging and the fairness threshold, as measurable and traceable requirements.

IEEE 830-style SRS for HireSense. Every requirement carries an acceptance criterion and traces to the module that implements it and the test that verifies it.

Contents

1	Introduction	2	Overall description
3	Requirement elicitation and analysis	4	Functional requirements
5	Non-functional requirements	6	Use cases
7	Requirements traceability matrix	8	Feasibility and validation

AUTHOR

Anshuman Atrey
Roll 150096724029

COURSE

Software Engineering & Project
Management
B.Tech CSE · Batch 2024–28 · Semester
V

DOCUMENT

D2 · v1.0
1 October 2026 · ITM Skills University

1 Introduction

1.1 Purpose

This Software Requirements Specification defines the requirements for **HireSense**, the rebuilt applicant-screening system of a staffing company that screens **90,000 applications a year**. It follows the structure of IEEE Std 830-1998. It is written so that every requirement is *testable*: each one has a measurable acceptance criterion and traces to a design element and a test (§7).

1.2 Scope

HireSense ingests PDF résumés, parses them with a confidence score, screens them with the specified routine against explicit per-job criteria, ranks candidates by explicit keys, logs every decision append-only, and runs a recurring bias audit with a defined consequence on breach. Out of scope: interview scheduling, offers, payroll, and candidate-facing job boards.

1.3 Problem statement (from the case study)

The current tool ranks candidates and recruiters trust the ranking. Nobody can explain why candidate 412 ranked above candidate 87. An internal review found the ranking correlated with the applicant's university tier far more strongly than with later job performance. Résumé parsing works on 71% of documents and fails silently on the rest.

1.4 Definitions

TERM	DEFINITION
Parse confidence	Weighted share of decision fields extracted reliably, × text-layer factor (1, or mean OCR word confidence). Range 0–1.
Criteria version	Immutable snapshot of a job's mandatory skills, bands, threshold, city and remote flag.
Selection rate (SR)	Shortlisted ÷ decided applications for a group in an audit window.
Impact ratio	SR of a group ÷ highest SR among comparable groups (4/5ths rule when ≥ 0.80).
Declared group	Group from voluntary self-identification (gender, age band), or derived for audit only (university tier).
Silent failure	A parse that is wrong but has confidence ≥ 0.6 , so it is decided automatically.
Shadow mode	The legacy system runs on the same input for comparison but never affects decisions.

1.5 References

- Case Study 31, SEPM, B.Tech CSE (given data used verbatim).
- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
- I. Sommerville, *Software Engineering* (prescribed text): requirements engineering and validation.
- Uniform Guidelines on Employee Selection Procedures, 29 CFR §1607.4(D): the four-fifths rule.
- Digital Personal Data Protection Act, 2023 (India).

2 Overall description

2.1 Product perspective

HireSense replaces the legacy screener without stopping screening: it runs in **shadow mode** beside the legacy system until each increment's exit criteria are met (Project Plan, D5). It is a web application (Next.js) with a screening API (Bun + Hono), a SQLite store, an optional on-device OCR model and an optional local language model.

2.2 Stakeholders and user classes

STAKEHOLDER	INTEREST	INFLUENCE	KEY NEEDS
Recruiter	Fast, defensible shortlists	High	Ranked list with reasons; trust that unreadable résumés are not dropped
Reviewer (talent ops)	Clear manual-review queue	Medium	See why an application was routed; correct fields quickly
Criteria owner / hiring manager	Hire the right people	High	Configure criteria without engineers; preview impact
Compliance / audit lead	Defensible, non-discriminatory process	High	Recurring audit, evidence, defined breach response
Head of Talent	Reputation, throughput	High	Escalations, KPIs
Candidates	Fair and explainable treatment	Low (individually) · high (collectively, reputational)	Not rejected because a parser failed; data protection
Engineering	Maintainable system	Medium	Isolation of parsing and ranking; tests; versioning

2.3 Product functions

- Ingest résumé PDFs; parse with confidence; OCR fallback for scans.
- Configure, version and preview ranking criteria per job.
- Screen with the specified routine; rank by explicit keys; explain any decision or ordering.
- Append-only decision log with search and CSV export.
- Manual review queue with optional AI-assisted extraction.
- Recurring bias audit with severity grading and automatic consequence.

2.4 Constraints

- Use the case-study figures as given: the routine, the 0.6 confidence gate, the 29% failure rate, and the 80% rule.
- Screening must continue during the rebuild.
- Self-declared attributes must never be inputs to screening.
- Runs on commodity hardware (reference: 8 GB laptop).

2.5 Assumptions and dependencies

- Self-identification is voluntary. A non-trivial share answers "Prefer not to say" (85 of 2,000 in the demo data).
- University tiers are an illustrative list used only to audit for the proxy the internal review found.
- All candidate data in the prototype is synthetic.

3 Requirement elicitation and analysis

Requirements were elicited by **document analysis** of the case study and its internal-review findings, by **scenario analysis** of the failure modes (silent parse failure, unexplainable ranking, tier correlation), and by **prototyping**: a working prototype was built and measured against ground truth, and it exposed requirements the text did not state, such as the anomaly cap and the minimum group size.

PROBLEM IN THE CASE STUDY	ROOT CAUSE	REQUIREMENT RESPONSE
Nobody can explain why 412 ranked above 87	Opaque score; implicit ordering	FR-S1-S3: specified routine, stored trace, explicit rank keys
Ranking tracked university tier	Tier used directly or by proxy	FR-S4 (excluded inputs) + FR-F1-F4 (audit catches proxies)
Parsing fails silently on 29%	No confidence; failure looks like 'no skills'	FR-P1-P4: confidence, routing, OCR, acceptance threshold
Rejections not traceable	No reason logging	FR-L1-L3: append-only log with reasons

4 Functional requirements

4.1 Résumé parsing

The case study states parsing works on 71% of documents and fails silently on the rest. These requirements make every parse result carry its own reliability, and make failure visible.

ID	REQUIREMENT	ACCEPTANCE CRITERION (MEASURABLE)	PRIORITY
FR-P1 Parse with confidence	The system shall extract skills, years of experience, city, name and email from a PDF résumé and attach a parse confidence in [0, 1] with a per-field breakdown.	Every stored parse has confidence $\in [0,1]$ and a breakdown of 5 weighted fields summing to 1.	Must
FR-P2 No silent failure	A résumé with no usable text layer (< 200 characters) or with a recognised section that yields no data shall receive confidence < 0.6 and must never be rejected automatically.	0 automated rejections of documents with confidence < 0.6 (decision log query).	Must
FR-P3 OCR fallback	Where the PDF has no text layer, the system should recover text by OCR and scale confidence by the mean OCR word confidence.	Scanned-résumé parse success $\geq 95\%$ on the evaluation corpus.	Should
FR-P4 Parse acceptance threshold	Released parser versions shall correctly parse $\geq 95\%$ of the evaluation corpus with $\leq 1\%$ silent failures (wrong but confidence ≥ 0.6).	Measured by seed/evaluate.ts against ground truth on every build.	Must

4.2 Configurable ranking criteria

Ranking criteria must be explicit and configurable. Criteria are data, never code.

ID	REQUIREMENT	ACCEPTANCE CRITERION (MEASURABLE)	PRIORITY
FR-C1 Explicit criteria per job	Each job shall have explicit criteria: mandatory skills, experience bands with scores, shortlist threshold, job city and a remote-allowed flag.	Criteria schema validated on save; invalid criteria rejected with a message.	Must
FR-C2 Versioned changes	Changing criteria shall create a new immutable version with author, timestamp and a mandatory change note; every decision shall reference the version it used.	100% of decision_log rows carry a criteria_version_id.	Must

ID	REQUIREMENT	ACCEPTANCE CRITERION (MEASURABLE)	PRIORITY
FR-C3 Impact preview	Before publishing, a recruiter should be able to dry-run draft criteria on all applicants of the job and see outcome shifts and the fairness check.	Preview returns outcome counts and impact ratios without writing to the log.	Should
FR-C4 Ranking change isolation	A change to a ranking rule shall not require any change to the parsing module.	ranking/ and scoring/ import nothing from parsing/ (static dependency check).	Must

4.3 Screening and explainability

Nobody could explain why candidate 412 ranked above candidate 87. Every decision and every rank order must be explainable from stored facts.

ID	REQUIREMENT	ACCEPTANCE CRITERION (MEASURABLE)	PRIORITY
FR-S1 Specified routine	Screening shall implement the routine exactly as given: missing mandatory skills → reject; else score experience band; shortlist if score ≥ threshold and location matches, else if score ≥ threshold and remote allowed; else waitlist; route to manual review if parse confidence < 0.6.	All 8 CFG paths produce the specified outcome (D1).	Must
FR-S2 Decision explanation	Every decision shall store a reason code, a human-readable reason and the trace of CFG nodes visited with the values tested at each.	100% of decisions have non-empty reason and a trace ending at N9/N10.	Must
FR-S3 Explicit rank order	Candidates of a job shall be ordered by explicit keys only: outcome, then band score, then application time, then application id; the system shall answer "why is A above B" with the first differing key.	Compare view returns winner and key for any two applications.	Must
FR-S4 Protected attributes excluded	Screening and ranking shall not read university, name, email or any self-declared attribute.	Metamorphic tests: changing these leaves outcome and score unchanged.	Must

4.4 Audit logging

Every automated rejection must be logged with a reason.

ID	REQUIREMENT	ACCEPTANCE CRITERION (MEASURABLE)	PRIORITY
FR-L1 Log every decision	Every automated and human decision shall be appended to a decision log with outcome, tentative outcome, reason code, reason, score, criteria version, parser version, confidence, trace, actor and timestamp.	count(decision_log) ≥ count(applications); every rejection has reason_code.	Must
FR-L2 Append-only	The decision log shall be append-only: update and delete shall be refused by the storage layer.	SQL UPDATE/DELETE on decision_log raise an error.	Must
FR-L3 Export	Authorised users shall be able to filter the log (outcome, reason, job, actor) and export it as CSV.	CSV export contains every filtered row with 14 columns.	Should

4.5 Bias audit

Selection rate for any declared group must not fall below 80% of the highest group's rate.

ID	REQUIREMENT	ACCEPTANCE CRITERION (MEASURABLE)	PRIORITY
FR-F1 Fairness rule (testable form)	For every declared group g with $n_g \geq 30$ decided applications in the audit window, $SR_g \geq 0.80 \times \max_h SR_h$, where $SR = \text{shortlisted} \div \text{decided}$. A group below this is a breach.	0.80 passes, 0.78 breaches; groups with $n < 30$ reported but not enforced.	Must
FR-F2 Recurring audit	The audit shall run monthly over a rolling 90-day window per job and company-wide, after every criteria change, and on demand.	One scheduled audit per scope per month in the audits table.	Must
FR-F3 Breach consequence	A statistically significant breach ($z \geq 1.96$) shall pause automated rejection for the job until a re-audit passes; a non-significant breach shall open a warning incident.	After a critical audit, new rejections for the job are routed to manual review.	Must
FR-F4 Self-identification	Declared groups shall come from voluntary self-identification stored separately from screening data, with "Prefer not to say" excluded from comparison.	self_id is read only by audit; 'Prefer not to say' has no ratio.	Must

4.6 Human review

Low-confidence parses go to a person.

ID	REQUIREMENT	ACCEPTANCE CRITERION (MEASURABLE)	PRIORITY
FR-R1 Review queue	Applications in manual review shall be listed oldest-first with the reason they were routed and the withheld automated outcome.	Queue shows reason code, confidence and tentative outcome per row.	Must
FR-R2 Verified re-screen	A reviewer shall correct the parsed fields; the same routine then decides with confidence 1, and the log records the reviewer as actor.	Reviewer decisions have actor 'reviewer:<name>'.	Must
FR-R3 AI-assisted extraction	A local small language model could propose field values; any value not found in the résumé text shall be discarded.	Hallucinated skills, cities and employers are dropped.	Could

4.7 Fairness requirement as an executable test

Given 100 decided applicants in group A with 50 shortlisted, and 100 in group B with 40 shortlisted, **when** the audit runs, **then** B's impact ratio is $0.40 / 0.50 = 0.80$ and the audit passes.

Given B instead has 39 shortlisted, **then** the ratio is 0.78, a **breach** is reported for B, and the job's severity is graded by significance (FR-F3).

Given B has only 10 decided applicants, **then** B is reported as low-sample and cannot trigger a breach.

5 Non-functional requirements

REQUIREMENT	SCOPE	TARGET	EVIDENCE
NFR-1 Performance	Screening decision for a parsed application	≤ 50 ms p95 (routine is $O(\text{skills})$); $OCR \leq 3$ s per page	Routine < 1 ms per call in unit tests; OCR 237 ms/page on an M2 laptop
NFR-2 Throughput	Volume from the case study	90,000 applications/year ≈ 360 per working day (250 days); peak $5 \times = 1,800$ /day	Seed of 2,000 parses + screens in ≈ 4 min including OCR
NFR-3 Availability	Screening during migration	Screening never stops: legacy remains primary until each increment's exit criteria are met (D5)	Shadow mode runs both systems on the same PDFs

REQUIREMENT	SCOPE	TARGET	EVIDENCE
NFR-4 Auditability	Decision records	100% of decisions reproducible from (parse, criteria version, routine version)	Append-only log; versions on every row
NFR-5 Privacy	Personal & self-ID data	Self-ID visible to audit role only; never shown to recruiters; data minimised (DPDP Act 2023)	Separate table; not returned by application APIs
NFR-6 Maintainability	Rule change effort	A ranking rule change touches 0 files in parsing/; criteria change needs no deploy	Static import check; criteria are data
NFR-7 Usability	Recruiter task	Explain any decision in ≤ 2 clicks from the ranked list	Candidate page shows path, reason, rank keys
NFR-8 Portability	Deployment	Runs on a single host with Bun; SQLite file DB; no external services required	OCR and AI optional with graceful fallback

6 Use cases

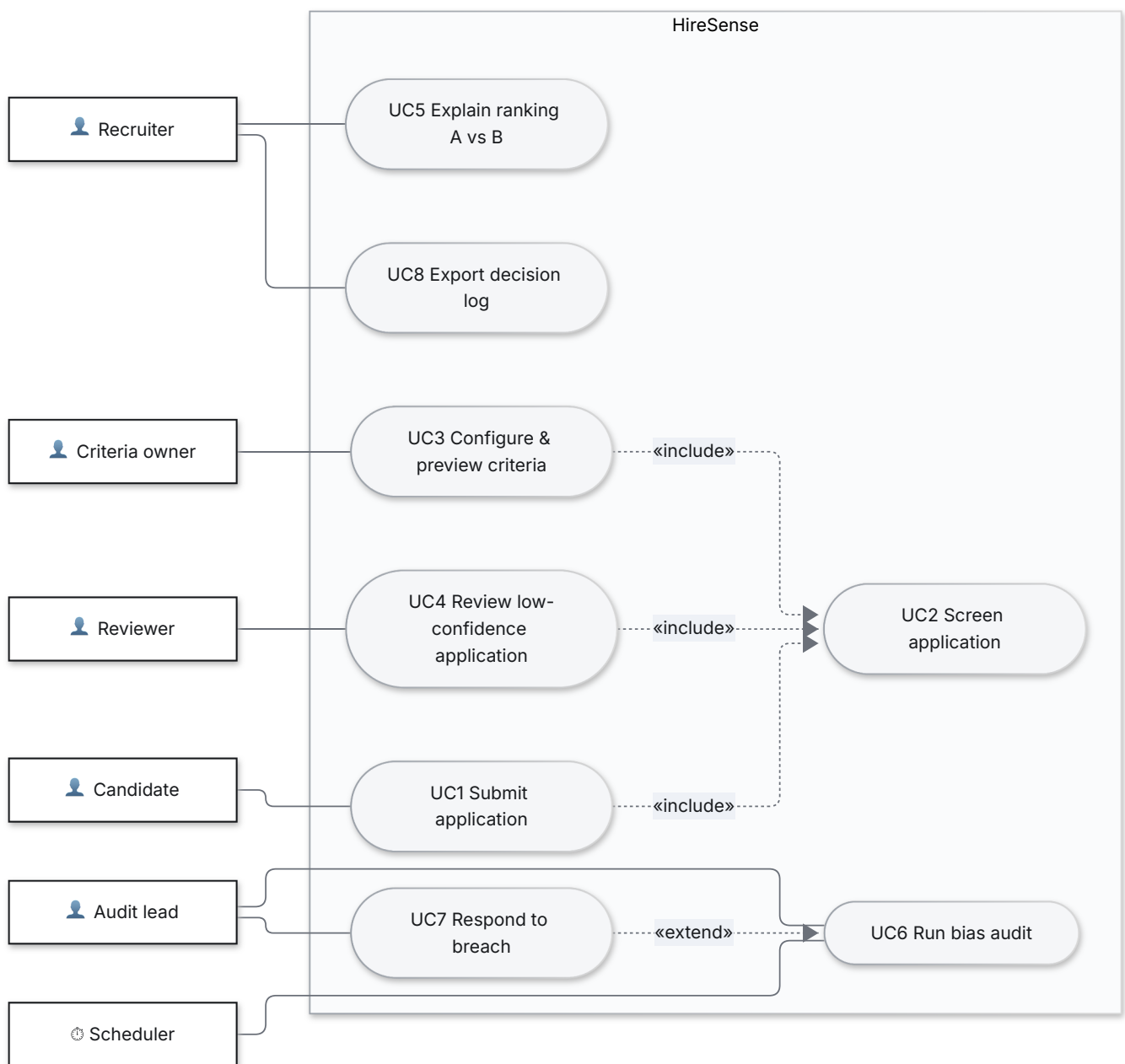


Figure 1. Use-case diagram. Screening (UC2) is included by submission, by review and by criteria publication. A breach response extends the audit.

UC	ACTOR	MAIN FLOW	ALTERNATIVE / EXCEPTION	REQS
UC1 Submit application	Candidate	Upload PDF + optional self-ID → stored → UC2	Not a PDF / > 5 MB → rejected with message	FR-P1, FR-F4
UC2 Screen application	System	Parse → (OCR if no text) → routine → policy → log → rank	Confidence < 0.6 → manual review; job paused → rejections to review	FR-P2, FR-S1, FR-L1
UC3 Configure criteria	Criteria owner	Edit → preview impact → publish with note → re-screen job → audit	Invalid bands/skills → error; no note → blocked	FR-C1–C3, FR-F2
UC4 Review application	Reviewer	Open queue → compare PDF and fields → (AI suggest) → confirm → routine decides	Model unavailable → manual entry	FR-R1–R3
UC5 Explain ranking	Recruiter	Pick A and B → first differing rank key + both paths	Different jobs → compare outcomes, not ranks	FR-S2, FR-S3
UC6 Run bias audit	Scheduler / audit lead	Monthly / on change / on demand → compute ratios → grade → record	Insufficient sample → reported only	FR-F1, FR-F2
UC7 Respond to breach	Audit lead	Warning → incident; critical → pause auto-reject → remediate → re-audit	Re-audit passes → resume	FR-F3
UC8 Export log	Recruiter / audit lead	Filter → CSV	—	FR-L3

7 Requirements traceability matrix

Each requirement traces forward to the design element that implements it and the automated test that verifies it (test references are `file:line` in `api/test/`). Requirements verified at system level are verified through the API and the review checklist.

REQ	SOURCE	DESIGN ELEMENT	VERIFIED BY	STATUS
FR-P1	CS: parsing quality	parsing/parser.ts · WEIGHTS	parser.test.ts:20 parser.test.ts:148	Verified
FR-P2	CS: 29% fail silently	parser.ts · MIN_TEXT_CHARS, ANOMALY_CAP	parser.test.ts:46 parser.test.ts:56	Verified
FR-P3	Work package WP-P	parsing/ocr.ts (PP-OCrv6 tiny)	parser.test.ts:131	Verified
FR-P4	CS: define acceptance threshold	seed/evaluate.ts	parser.test.ts:68 parser.test.ts:77	Verified
FR-C1	CS: explicit and configurable	types.ts Criteria · index.ts validateCriteria	routine.test.ts:108 routine.test.ts:99	Verified
FR-C2	CS: explainability	criteria_versions table	API smoke / review	System test
FR-C3	Preventive maintenance	POST /jobs/:id/criteria/preview	API smoke / review	System test
FR-C4	CS outcome: modular design	module boundaries (D3 §2)	API smoke / review	System test
FR-S1	CS: ranking routine	ranking/routine.ts screen()	routine.test.ts:10 routine.test.ts:18 routine.test.ts:25 routine.test.ts:32 routine.test.ts:39	Verified
FR-S2	CS: explainability	Decision.trace · decision_log.trace	routine.test.ts:133	Verified
FR-S3	CS: candidate 412 vs 87	ranking/order.ts explainOrder()	audit.test.ts:81 audit.test.ts:88	Verified
FR-S4	CS: internal review (tier)	self_id table segregated	routine.test.ts:121 routine.test.ts:127	Verified

REQ	SOURCE	DESIGN ELEMENT	VERIFIED BY	STATUS
FR-L1	CS: log every rejection	decision_log table · screening.ts	API smoke / review	System test
FR-L2	Auditability	SQLite triggers	audit.test.ts:101 audit.test.ts:102	Verified
FR-L3	Audit evidence	GET /decisions.csv	API smoke / review	System test
FR-F1	CS: audit requirement	audit/fairness.ts	audit.test.ts:16 audit.test.ts:22 audit.test.ts:29	Verified
FR-F2	CS: periodic bias audit	audit/service.ts runAudit	audit-service.test.ts:40	Verified
FR-F3	CS: defined consequence	service.ts severityOf · screening policy	audit-service.test.ts:48 audit-service.test.ts:61 audit-service.test.ts:69	Verified
FR-F4	Privacy	self_id table	'Prefer not to say' is never compared	Check
FR-R1	CS: manual review	GET /review	API smoke / review	System test
FR-R2	Accountability	POST /applications/:id/review	API smoke / review	System test
FR-R3	Reviewer productivity	parsing/llm.ts	llm.test.ts:47 llm.test.ts:40	Verified

Table 1. Coverage of the matrix

22 functional requirements · 15 verified by automated unit/integration tests · 7 verified by system tests through the API. Non-functional requirements are verified by measurement (§5).

8 Feasibility and validation

FEASIBILITY	ASSESSMENT
Technical	Feasible: a working prototype implements every Must requirement. Measured 99.0% parse success and 6 silent failures in 2,000 résumés; all automated tests pass.
Operational	Feasible: shadow mode and increments let recruiters keep working; the review queue load fell from 215 to 94 after OCR.
Economic	Open-source stack (Bun, Next.js, SQLite, PP-OCR, Qwen3 1.7B), no licence fees; runs on existing laptops/servers. Effort estimate in D5.
Legal / ethical	Self-ID is voluntary and segregated; adverse-impact monitoring follows the four-fifths rule; personal data minimised (DPDP Act 2023).

Requirement validation checklist

CHECK	RESULT
Every requirement has a unique ID and a measurable acceptance criterion	Yes
Case-study figures used verbatim (routine, 0.6, 29%, 80%)	Yes
Design parameters we chose are labelled as such ($n \geq 30$, $z \geq 1.96$, 90-day window, weights)	Yes
No requirement conflicts (e.g. 'never reject a low-confidence parse' vs routine): resolved by N9 overriding to manual review	Yes
Each Must requirement is traced to design and verification	Yes



D3 · DELIVERABLE

Design Pack

Architecture, module cohesion justification, class, activity, sequence, state, deployment and data-model diagrams, and the user interface.

How HireSense separates parsing, scoring and ranking into cohesive modules so that a ranking-rule change never touches parsing, shown with UML views and a dependency graph measured from the code.

Contents

1	Architecture overview	2	Module decomposition, cohesion and coupling
3	Class diagrams	4	Activity diagram: screening pipeline
5	Sequence diagrams	6	State diagrams
7	Deployment and data model	8	User interface

AUTHOR	COURSE	DOCUMENT
Anshuman Atrey	Software Engineering & Project	D3 · v1.0
Roll 150096724029	Management	1 October 2026 · ITM Skills University
	B.Tech CSE · Batch 2024–28 · Semester	
	V	

1 Architecture overview

HireSense is a layered, modular system. A web client (Next.js) talks to a screening API (Bun + Hono). The API is composed of single-purpose modules around one pure decision function. Parsing, scoring, ranking and auditing are separate modules that communicate only through plain data types (`ParsedResume` , `Criteria` , `Decision`).

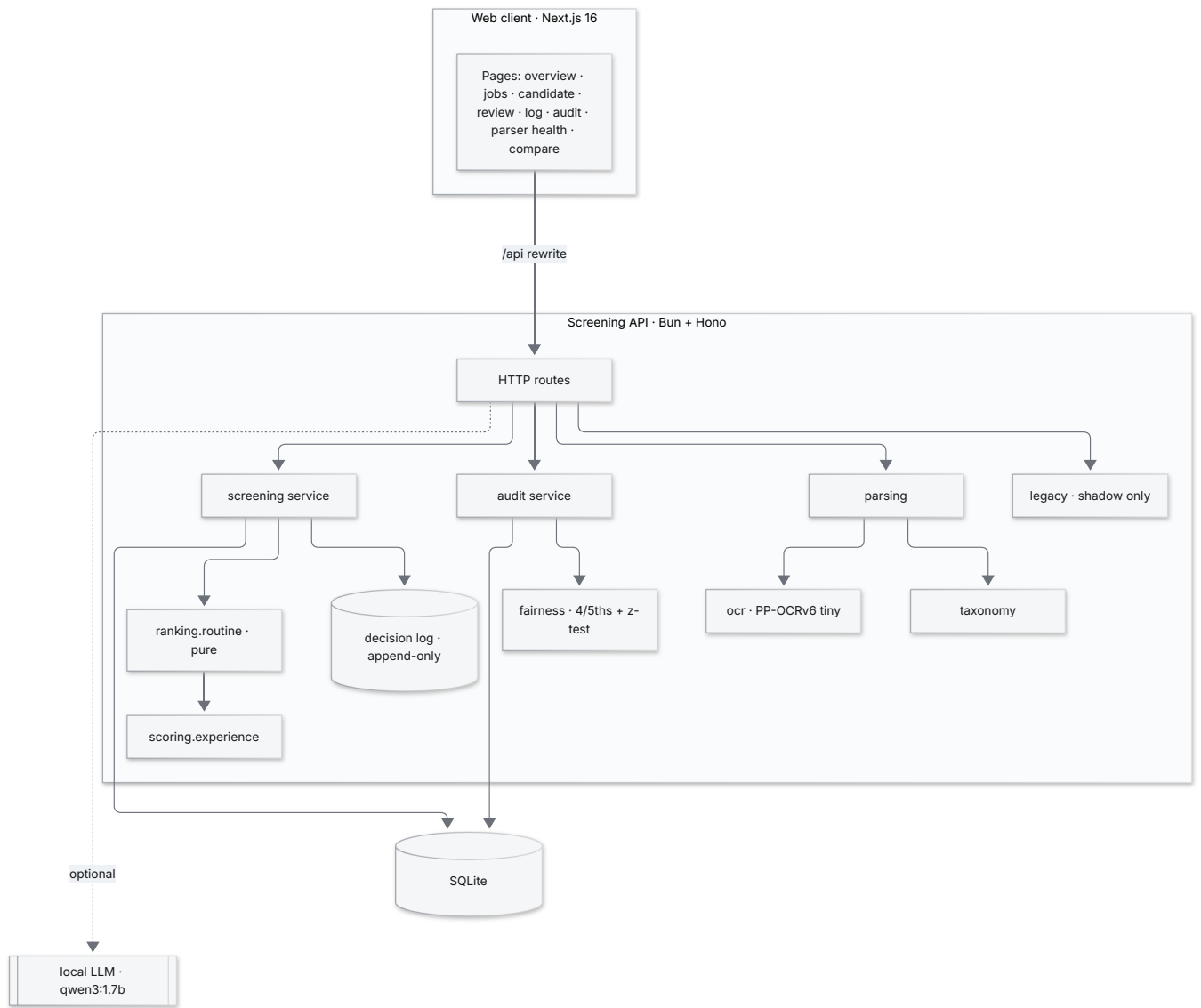


Figure 1. Component view. Dashed: optional dependency with graceful fallback.

DECISION	CHOICE	WHY
Criteria representation	Versioned data rows, not code	Recruiters change rules without a deploy; every decision references the version it used (explainability)
Decision core	One pure function per the specified routine	Directly testable (white-box); the CFG is read from the code; the trace is free
Log integrity	SQLite triggers refuse UPDATE/DELETE	The storage layer enforces append-only, not only the application
Migration	Shadow mode (legacy runs on the same PDFs)	Screening never stops; improvement is measured, not claimed
Parsing reliability	Confidence + gate + OCR fallback	Turns silent failure into routed, explained review
AI use	Local small model for extraction only, grounded in text	Never decides; hallucinated values are discarded

DECISION	CHOICE	WHY
Storage	SQLite file	Zero-ops for the prototype; schema portable to Postgres

2 Module decomposition, cohesion and coupling

The case study asks that **parsing, scoring and ranking** be separate cohesive modules, so that **changing a ranking rule does not touch parsing**. The table gives each module's single responsibility and its cohesion type (Sommerville; the Yourdon & Constantine scale).

MODULE	RESPONSIBILITY	COHESION	COUPLING TO OTHERS
parsing/	PDF → text items → reading order → sections → fields → confidence; OCR fallback	Functional one transformation	Produces ParsedResume (data coupling)
scoring/	Years of experience → band score	Functional	Reads Band[] (data)
ranking/routine	The specified screening routine; returns Decision + trace	Functional	ParsedResume + Criteria in, Decision out (data)
ranking/order	Explicit rank keys; explain A vs B	Functional	Rankable records (data)
audit/fairness	Selection rate, impact ratio, z-test for one dimension	Functional	Rows of (outcome, group) (data)
audit/service	Windows, severity, breach action, persistence	Sequential pipeline	DB + fairness
screening.ts	Load parse + criteria → routine → policy → log	Sequential	DB + routine; the only writer of decisions
legacy/	Reconstructed old parser/ranker for shadow comparison	Functional	Read-only use of pdf + taxonomy

Measured dependency graph

Generated from the static imports in `api/src` at build time:

MODULE	IMPORTS FROM
audit	db, parsing, types
index	audit, db, legacy, parsing, ranking, screening, types
legacy	parsing
parsing	types
ranking	scoring, types
scoring	types
screening	db, ranking, types

Isolation check: passes. ranking and scoring import only `types` (and ranking imports scoring), with no dependency on `parsing`. A ranking rule change is a new criteria *version* (data) or an edit inside `ranking/`, and neither can affect parsing.

The one cross-module read is `audit → parsing/taxonomy`, for the university-tier lookup the audit uses. It is a read-only table (data coupling), and the routine never sees it.

No module uses **common coupling** (shared mutable globals) or **control coupling** (flags that steer another module's logic). Modules exchange only immutable records.

3 Class diagrams



Figure 2. Domain model. Composition marks ownership; every decision references the criteria version it used.



Figure 3. Service view. Dependencies point from parsing to OCR, and from screening to the routine and scoring. Nothing in scoring or ranking depends on parsing.

4 Activity diagram: screening pipeline

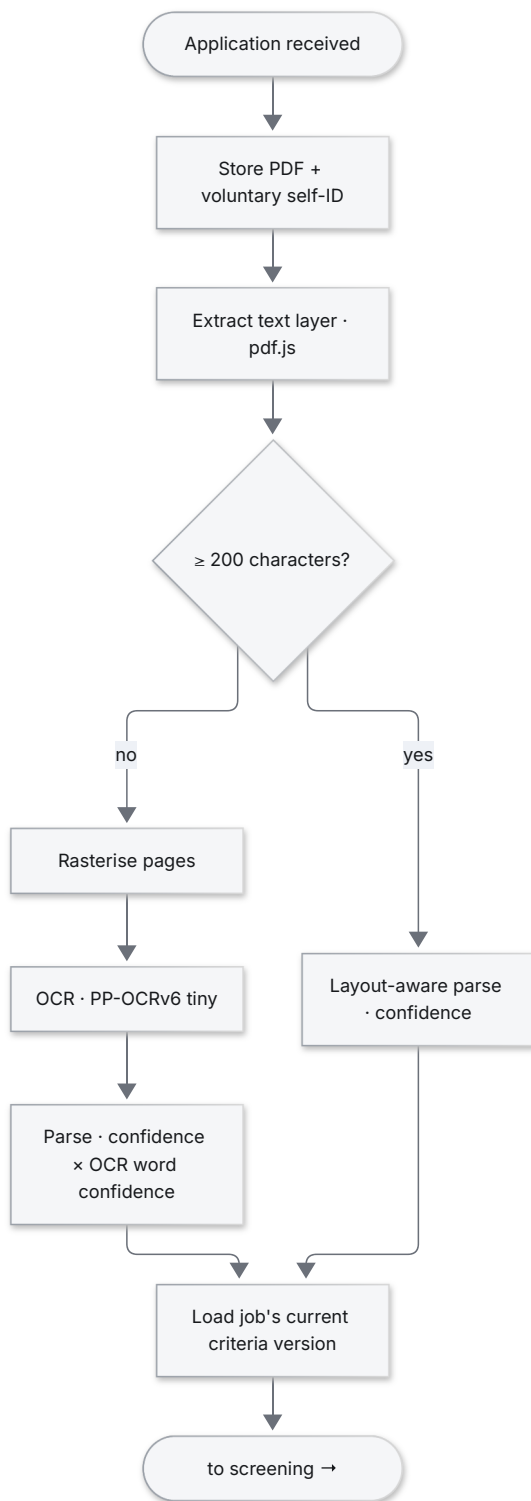


Figure 4. a. Ingestion and parsing.

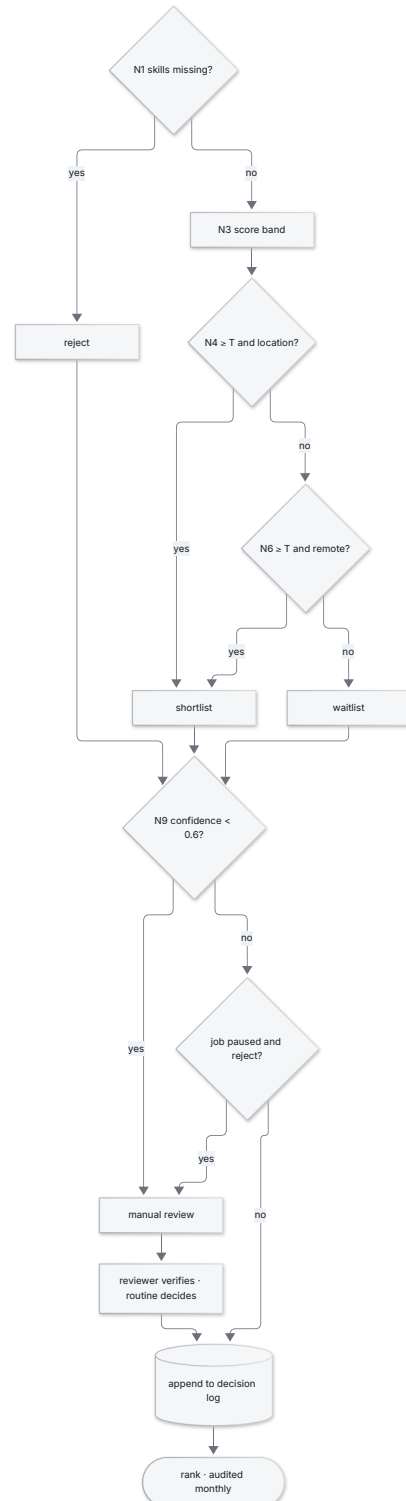


Figure 5. b. Screening, policy and review.

The routine (N1–N9) is exactly the specified one. The OCR fallback runs before it, and the pause policy after it, so neither changes the routine.

5 Sequence diagrams

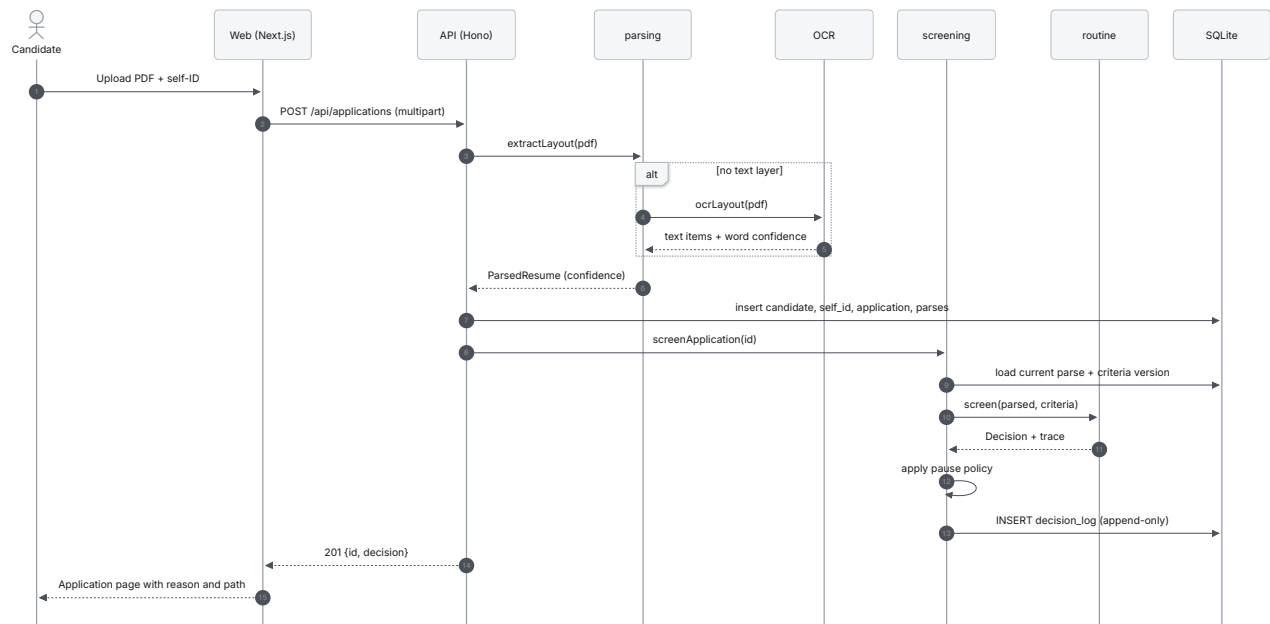


Figure 6. Sequence: submitting and screening an application.

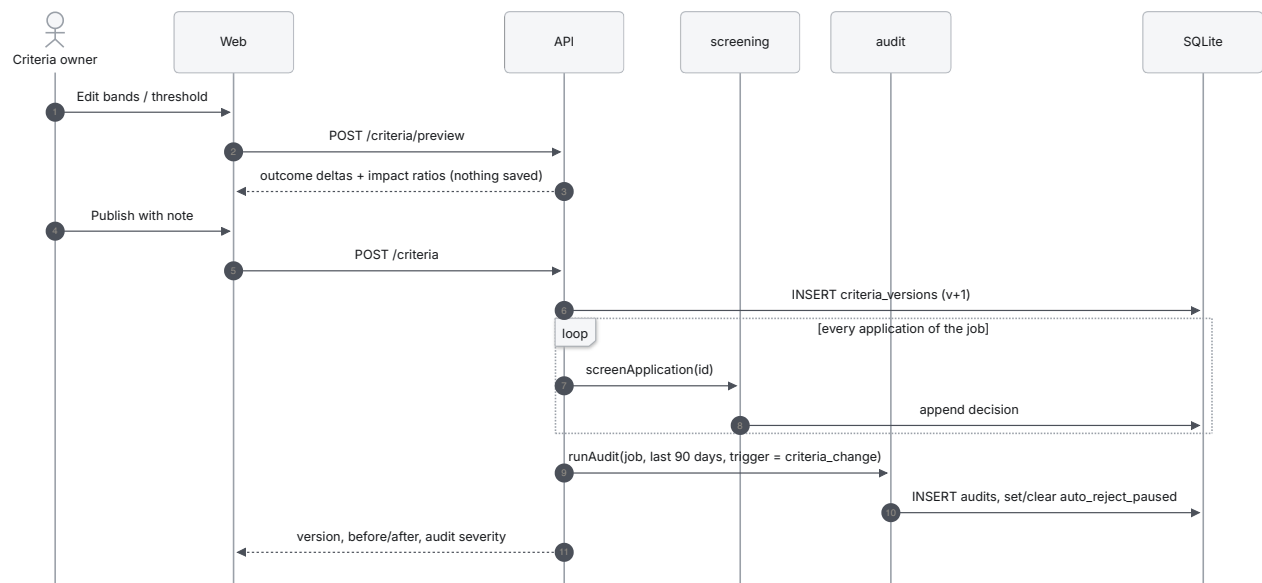


Figure 7. Sequence: changing criteria. Preview, publish, re-screen, then an automatic audit.

6 State diagrams

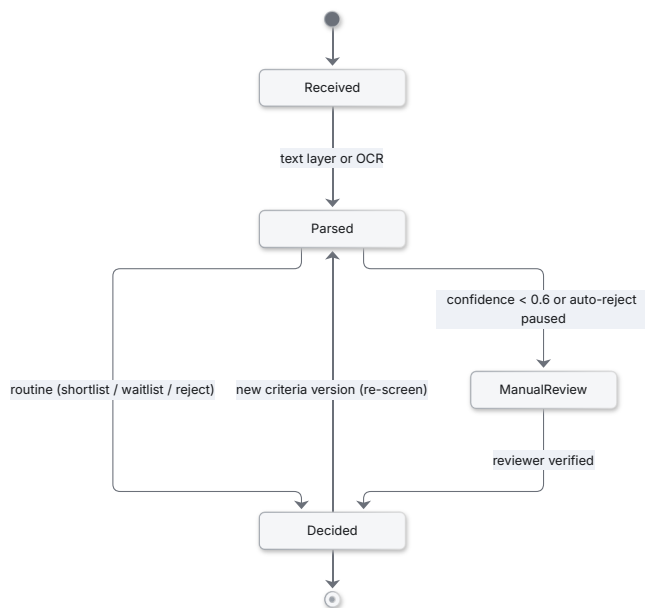


Figure 8. Application lifecycle.

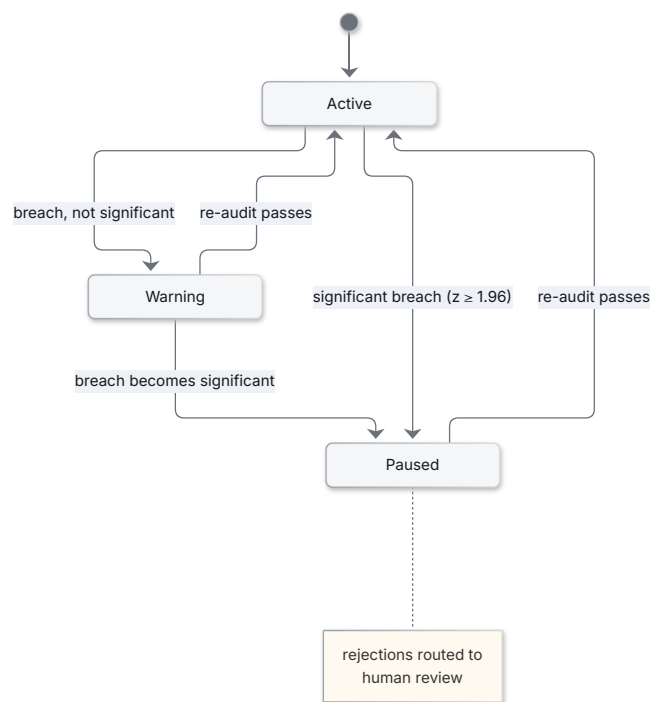


Figure 9. Job automated-rejection state, driven by audits.

7 Deployment and data model



Figure 10. Deployment diagram (prototype topology).

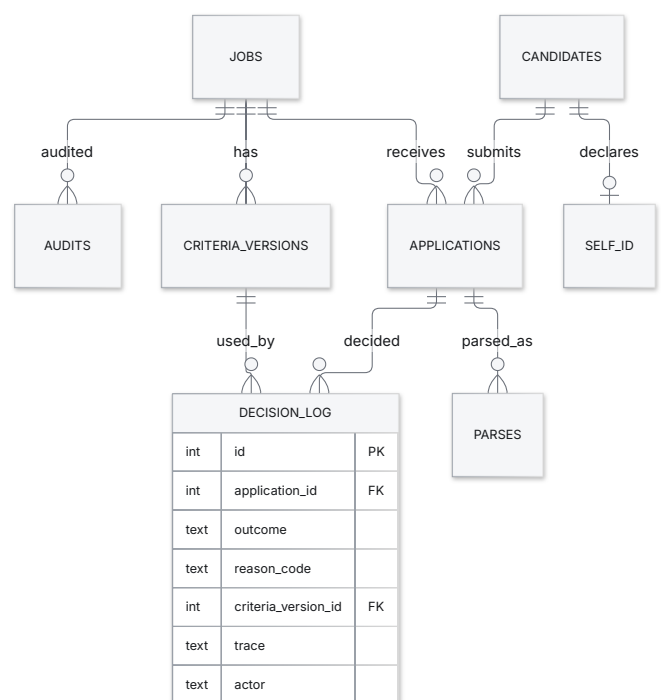


Figure 11. Entity-relationship model. decision_log is append-only (triggers).

8 User interface

The web client is built on **shadcn/ui** (the `dashboard-01` block, ★125k) with a minimal, monochrome style, and semantic colour reserved for outcomes. Screens below are from the running prototype with the seeded data.

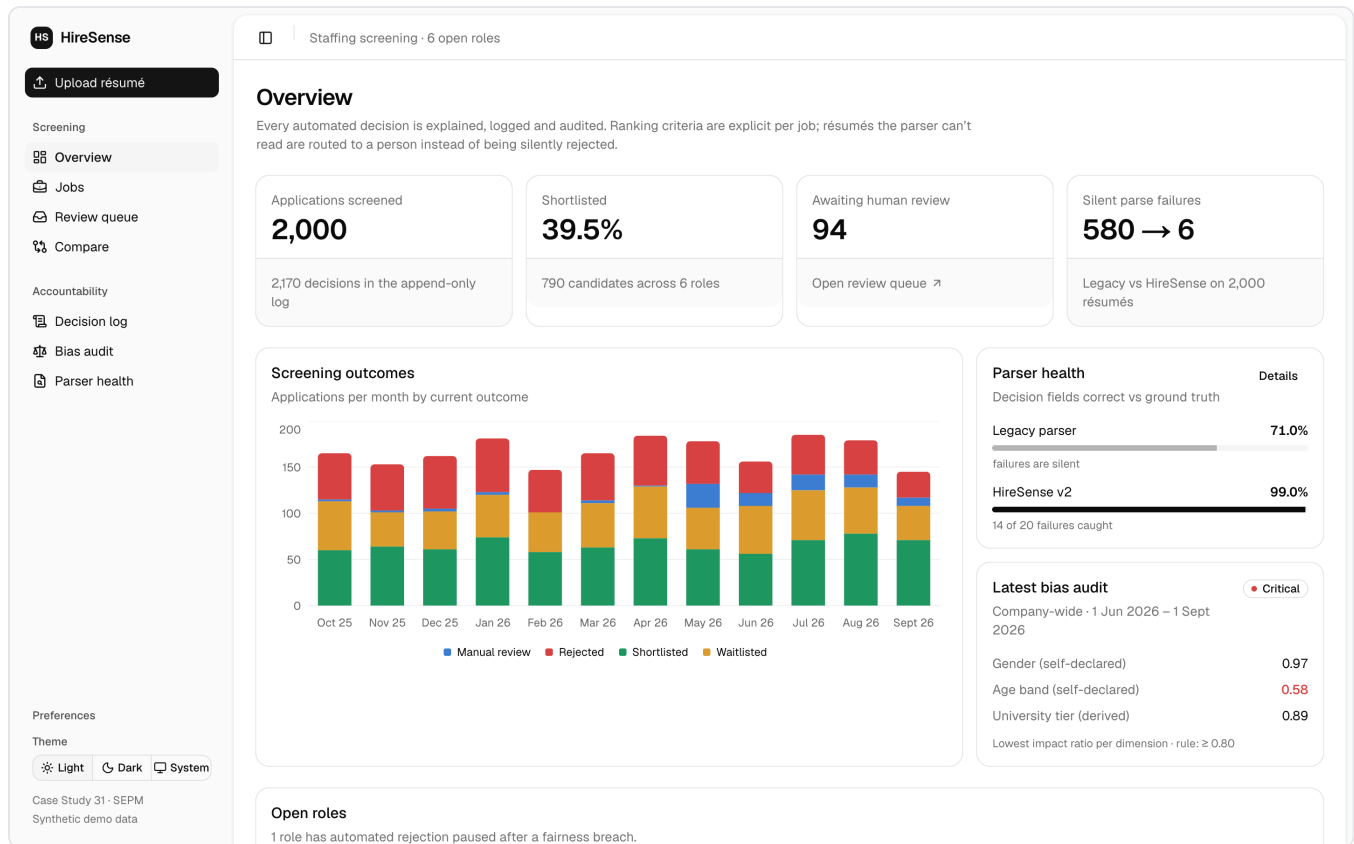


Figure 12. Overview: outcomes over time, parser health, latest audit, open roles.

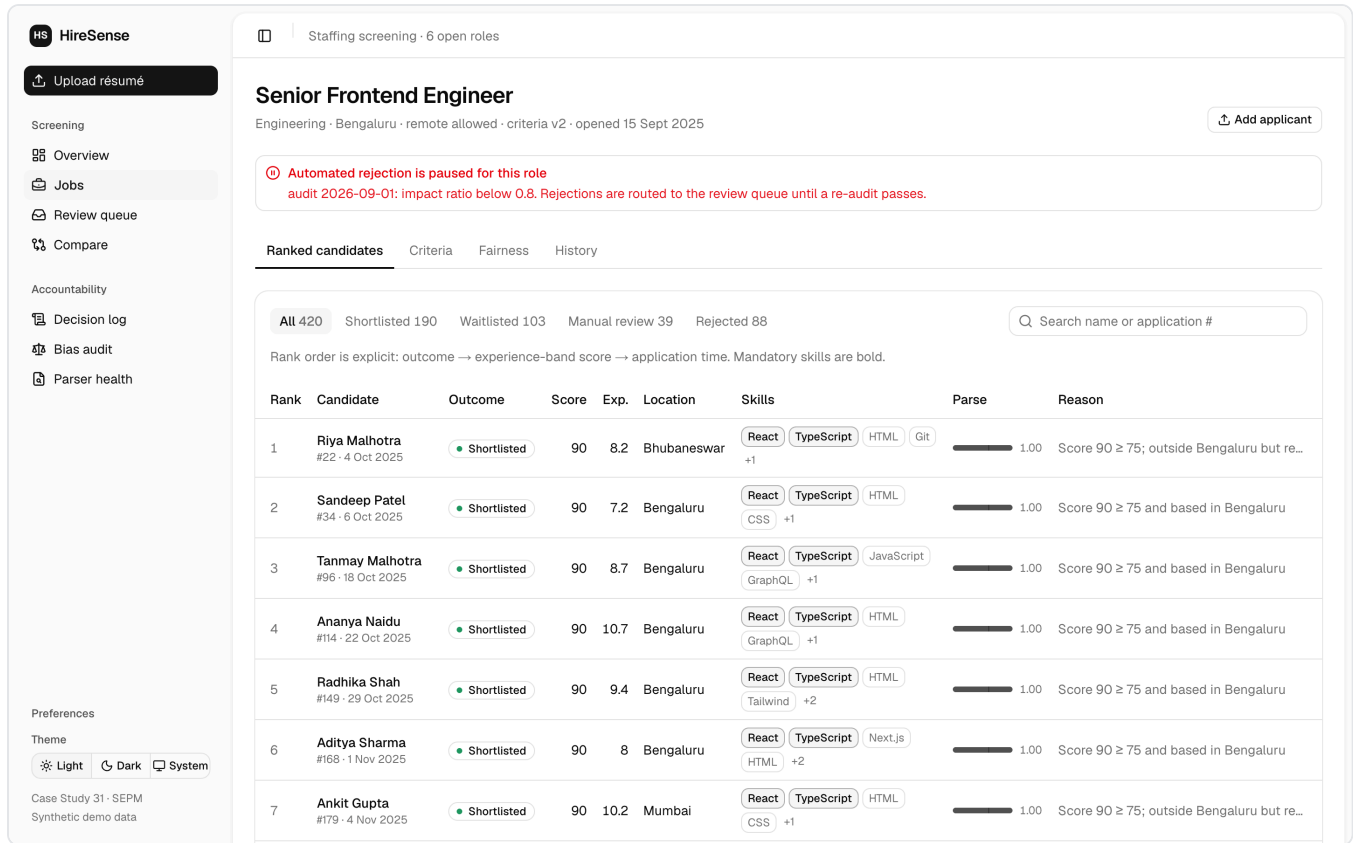


Figure 13. Ranked candidates for a role. Rank keys are explicit, and parse confidence is shown with the 0.6 gate.

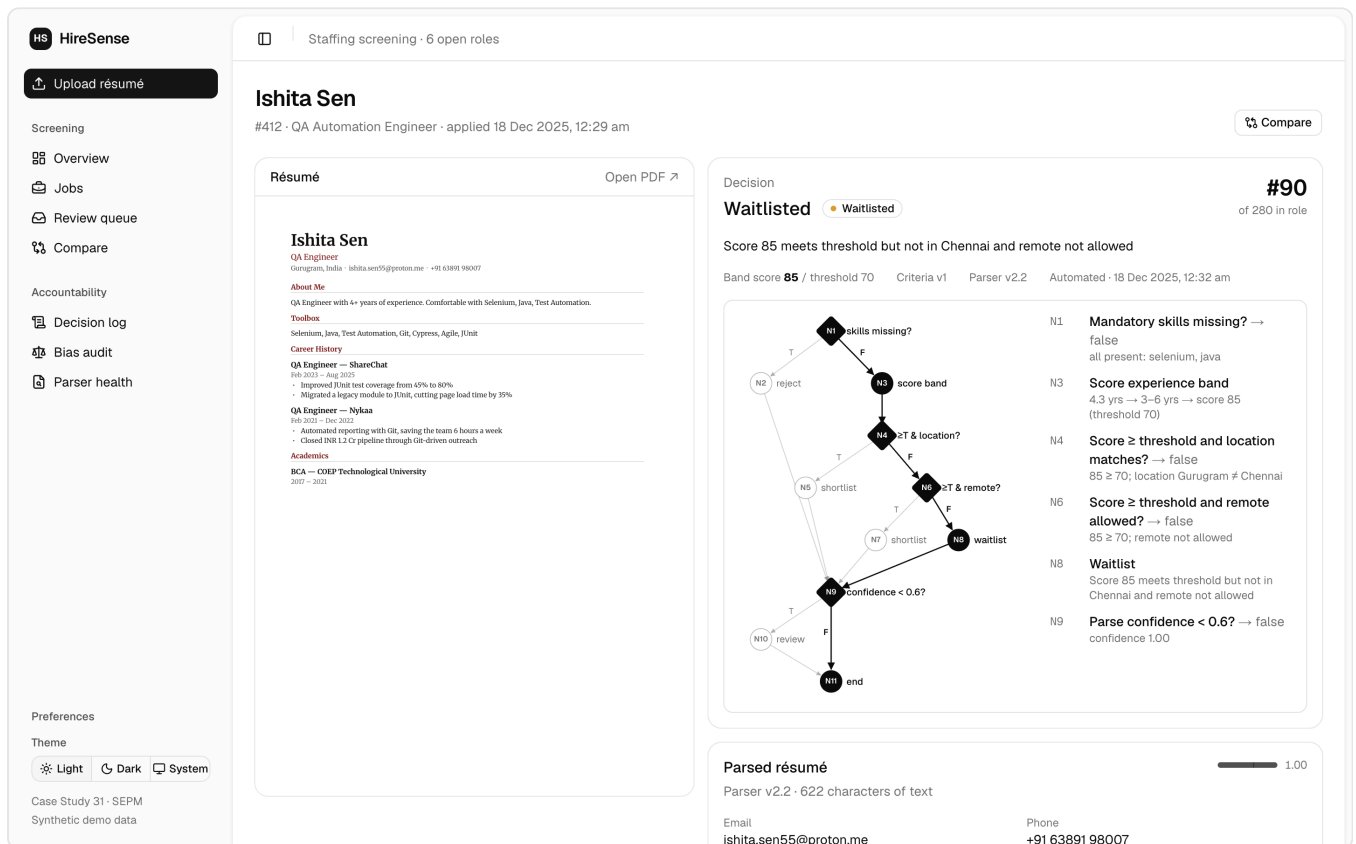


Figure 14. Candidate page: the résumé PDF beside the decision, its path through the CFG, parsed fields and the shadow legacy result.

HS HireSense

Upload résumé

Screening

Overview

Jobs

Review queue

Compare

Accountability

Decision log

Bias audit

Parser health

Preferences

Theme

LightDarkSystem

Case Study 31 - SEPM

Synthetic demo data

Staffing screening · 6 open roles

Senior Frontend Engineer

Engineering · Bengaluru · remote allowed · criteria v2 · opened 15 Sept 2025

Add applicant

Automated rejection is paused for this role

audit 2026-09-01: impact ratio below 0.8. Rejections are routed to the review queue until a re-audit passes.

Ranked candidatesCriteriaFairnessHistory

Ranking criteria

Editing v2. The screening routine is fixed; these values are the only inputs a recruiter can change.

Mandatory skills

Missing any of these → reject (N1 → N2).

React ×TypeScript ×Add skill

Experience bands

Years of experience → score (N3). Bands are contiguous from 0 years.

From (yrs)	To (yrs)	Score	
0	2	25	×
2	4	60	×
4	7	85	×
7	12	90	×
12	and above	80	×

Add band

Shortlist threshold

75

Impact preview

Dry run of the routine on every applicant for this role. Nothing is saved. Policy holds (a paused auto-reject) still apply on publish.

Change the criteria and run a preview to see outcome shifts and the fairness check before publishing.

Publish v3

Re-screens every applicant with the new version and runs a fairness audit.

Your name

e.g. Talent ops

Why is this changing?

Shown in the version history and audit trail

Publish and re-screen

Figure 15. Criteria editor: mandatory skills, bands, threshold, location and remote flag, with an impact preview before publishing.

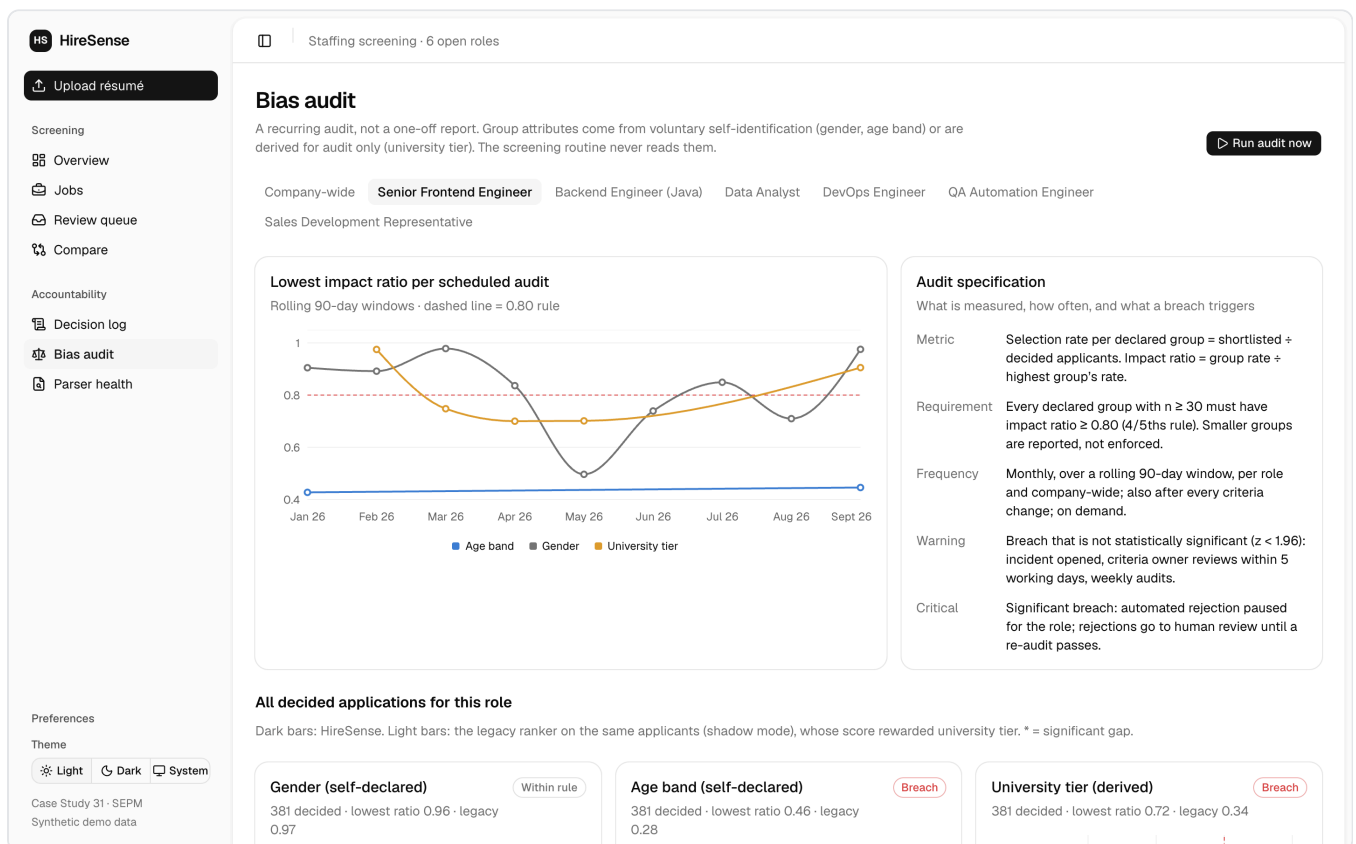


Figure 16. Bias audit: impact ratio per declared group, HireSense vs legacy, audit specification and history.



D4 · DELIVERABLE

Audit Specification

Metrics, measurement frequency, thresholds and the breach response procedure for the recurring bias audit.

Turns "selection rate for any declared group must not fall below 80% of the highest group's rate" into a measurable, repeatable audit with a defined consequence, and reports the current results.

Contents

1	Purpose and scope	2	Declared groups and data sources
3	Metrics	4	Requirement and acceptance tests
5	Frequency, windows and triggers	6	Thresholds and severity ladder
7	Breach response procedure	8	Current results (seeded data)
9	Limitations		

AUTHOR	COURSE	DOCUMENT
Anshuman Atrey	Software Engineering & Project	D4 · v1.0
Roll 150096724029	Management	1 October 2026 · ITM Skills University
	B.Tech CSE · Batch 2024–28 · Semester	
	V	

1 Purpose and scope

This specification defines the **recurring bias audit** of HireSense screening outcomes: what is measured, for which groups, how often, the thresholds, and the response procedure when a threshold is breached. It implements the case-study requirement:

"Selection rate for any declared group must not fall below 80% of the highest group's rate."

The audit covers automated and human screening decisions (shortlist / waitlist / reject) for every open role, and company-wide. It audits **outcomes**, not intentions. The screening routine never reads group attributes (D2 FR-S4), yet outcomes can still differ between groups through proxies. Catching that is the audit's purpose.

2 Declared groups and data sources

DIMENSION	GROUPS	SOURCE	HANDLING
Gender	Woman · Man · Non-binary · Prefer not to say	Voluntary self-identification at application	'Prefer not to say' is never compared
Age band	18–29 · 30–39 · 40+ · Prefer not to say	Voluntary self-identification	Same
University tier	Tier 1 · Tier 2 · Tier 3 · Unknown	Derived from the parsed university, for the audit only	Included because the internal review found the old ranking tracked tier; 'Unknown' not compared

Self-identification is stored in a separate `self_id` table. Only the audit module reads it. It is never shown to recruiters or returned by application APIs (D2 NFR-5).

3 Metrics

METRIC	DEFINITION	NOTES
Selection rate	$SR_g = \text{shortlisted}_g \div \text{decided}_g$	Decided = shortlist + waitlist + reject. Applications still in manual review are excluded until a person decides them.
Impact ratio	$IR_g = SR_g \div \max_{h \in C} SR_h$	C = comparable groups: $n \geq 30$ and not 'Prefer not to say'/'Unknown'. Small groups cannot set the benchmark.
Lowest impact ratio	$\min_{g \in C} IR_g$ per dimension	Reported as 'none' when fewer than two groups are comparable (insufficient sample).
Significance	$z = (SR_{\text{best}} - SR_g) \div \sqrt{(p(1-p)(1/n_g + 1/n_{\text{best}}))}$	Two-proportion z-test with pooled p ; $ z \geq 1.96 \Rightarrow$ the gap is unlikely to be sampling noise (5% level).
Sample size	n_g decided applications in the window	Groups with $n < 30$ are reported, not enforced.

4 Requirement and acceptance tests

Testable requirement (FR-F1)

For every declared group g in a dimension with $n_g \geq 30$ decided applications in the audit window: $IR_g \geq 0.80$. Any comparable group with $IR_g < 0.80$ is a **breach** of that dimension.

#	GIVEN	THEN	AUTOMATED TEST
AT-1	A: 100 decided, 50 selected · B: 100 decided, 40 selected	$IR_B = 0.80 \rightarrow$ pass (boundary is inclusive)	Pass
AT-2	A: 50/100 · B: 39/100	$IR_B = 0.78 \rightarrow$ breach	Pass
AT-3	A: 50/100 · B: 1/10	B is low-sample $\rightarrow$ reported, no breach	Pass
AT-4	A: 50/100 · 'Prefer not to say': 5/100	never compared	Pass
AT-5	A: 40 decided + 50 in manual review	rate uses the 40 decided only	Pass
AT-6	Only one group has $n \geq 30$	no ratio, 'insufficient sample'	Pass
AT-7	A: 20/40 · B: 15/40 (IR 0.75, not significant)	warning , job not paused	Pass
AT-8	A: 150/300 · B: 105/300 (IR 0.70, $z \geq 1.96$)	critical , auto-reject paused, the next rejection goes to review	Pass
AT-9	Paused job, re-audit passes	auto-reject resumes	Pass

AT-1 to AT-6 are in `audit.test.ts` and AT-7 to AT-9 in `audit-service.test.ts` (in-memory database, end to end).

5 Frequency, windows and triggers

TRIGGER	WHEN	SCOPE	WINDOW
Scheduled	1st of every month, 06:00	Each open role + company-wide	Rolling 90 days (enough sample per group)
Criteria change	Immediately after a new criteria version is published	That role	Last 90 days, re-screened under the new version
Pre-publication (preview)	Before publishing criteria	That role	All applicants, dry run (nothing recorded)
Parser / taxonomy release	After a new parser, OCR model or skill-taxonomy version	All roles	Last 90 days, re-parsed
On demand	Audit lead, complaint, or Head of Talent request	Any	Last 90 days
Escalated cadence	After a warning	That role	Weekly until two consecutive passes

Why a rolling 90-day window, not a calendar month? One role receives about 30 applications a month, which is too few for stable rates per group. In the demo data, monthly company-wide windows (~150 decisions) produced "critical" breaches from sampling noise alone. Ninety-day windows with $n \geq 30$ per group and a significance test separate real disparities from noise.

6 Thresholds and severity ladder

SEVERITY	CONDITION	AUTOMATIC ACTION	HUMAN ACTION AND DEADLINE
Pass	No comparable group below 0.80	Record audit; resume auto-reject if paused	—
Warning	Breach ($IR < 0.80$) but not significant ($z < 1.96$)	Open incident; raise this role's audit cadence to weekly	Criteria owner reviews the job-relatedness of the criteria within 5 working days
Critical	Breach and significant ($z \geq 1.96$)	Pause automated rejection for the role: every rejection is routed to human review	Audit lead and criteria owner start root-cause analysis within 2 working days ; Head of Talent informed

Company-wide breaches escalate to the Head of Talent and trigger per-role audits. They do not pause a role on their own.

7 Breach response procedure

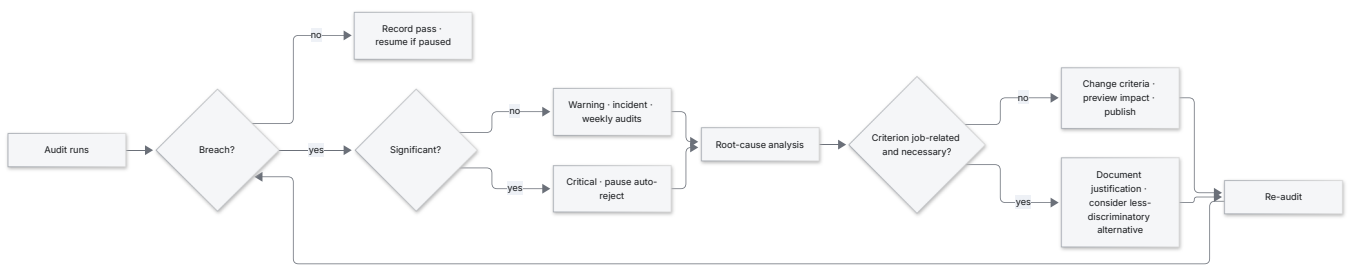


Figure 1. Breach response loop. Every step leaves a record in the audit table or the decision log.

STEP	WHAT HAPPENS	RESPONSIBLE	ACCOUNTABLE	RECORD
1 Detect	Scheduled or triggered audit computes ratios and grades severity	System	Audit lead	audits row (results, severity, action)
2 Contain	Critical: auto-reject paused for the role	System	Audit lead	jobs.auto_reject_paused + reason
3 Notify	Criteria owner and Head of Talent informed with the audit link	Audit lead	Head of Talent	Incident ticket
4 Analyse	Which criterion drives the gap? Compare by criterion; use the impact preview to test alternatives	Criteria owner	Audit lead	Analysis note
5 Remediate	Change criteria (new version, with a mandatory note), or document job-relatedness	Criteria owner	Head of Talent	criteria_versions row / justification
6 Verify	Re-audit (automatic on criteria change); two consecutive passes close it	System	Audit lead	audits rows
7 Review	Monthly governance review of open incidents and KPIs (D6)	Audit lead	Head of Talent	Minutes

8 Current results (seeded data)

AUDITS RECORDED 63 scheduled + triggered	LATEST COMPANY AUDIT critical 2026-06-01 → 2026-09-01	ROLES PAUSED 1 auto-reject paused now	REJECTIONS HELD FOR REVIEW 67 because of a pause
---	--	--	---

Per-role status (latest scheduled audit)

ROLE	WINDOW	SEVERITY	LOWEST RATIO	AUTOMATED REJECTION
Senior Frontend Engineer	2026-06-01 → 2026-09-01	Critical	0.45	Paused
Backend Engineer (Java)	2026-06-01 → 2026-09-01	Warning	0.73	Active
Data Analyst	2026-06-01 → 2026-09-01	Pass	1.00	Active
DevOps Engineer	2026-06-01 → 2026-09-01	Pass	0.94	Active
QA Automation Engineer	2026-06-01 → 2026-09-01	Pass	—	Active
Sales Development Representative	2026-06-01 → 2026-09-01	Pass	—	Active

Finding 1: the legacy ranker's university-tier bias is gone

Company-wide, over all decided applications, on the same applicants in shadow mode:

GROUP	N DECIDED	SELECTED	IMPACT RATIO	Z	LEGACY RATIO
Tier 1	240	45%	1.00	—	1.00
Tier 2	593	41%	0.90	1.17	0.48
Tier 3	1071	41%	0.90	1.28	0.23
Unknown (n < 30)	2	0%	—	—	—

The legacy score added 30 points for Tier 1 and 12 for Tier 2. HireSense does not read university, and the Tier-3 impact ratio moves from **0.23** (legacy) to **0.90**.

Finding 2: experience bands act as an age proxy on a senior role

Senior Frontend Engineer, age band, all decided applications:

GROUP	N DECIDED	SELECTED	IMPACT RATIO	Z	LEGACY RATIO
18–29	259	36%	0.46*	7.69	0.28
30–39	113	80%	1.00	—	1.00
Prefer not to say (n < 30)	9	67%	—	—	—

The routine never sees age. Yet a threshold that needs 4+ years of experience selects the 18–29 band at a much lower rate, and the gap is significant (*). The audit marked the role **critical** and paused automated rejection. The next step under §7 is for the criteria owner to show the experience requirement is job-related, or to lower it. The impact preview (dry run, computed for this report) shows that raising the 2–4 year band's score to the threshold (75) moves the age-band ratio from 0.46 to **0.78**: better, but still below 0.80, so the change alone would not close the incident. A real disparity that the routine itself cannot see is exactly what a recurring audit exists to find.

* significant at the 5% level ($z \geq 1.96$). All candidates and outcomes are synthetic; the age–experience relationship is a property of how the demo data was generated.

9 Limitations

- **Self-ID non-response.** Groups exist only for applicants who choose to declare. Non-response is reported, and if it grows the audit loses power.
- **Multiple comparisons.** 3 dimensions × several groups × 7 scopes per month mean some "significant" breaches occur by chance. The seeded history shows two such false alarms (D6 §7). A Holm–Bonferroni correction before grading critical is the planned improvement.
- **Intersectional groups** (e.g. women aged 40+) are not audited in v1; they need larger samples. This is planned as a perfective maintenance item (D6).
- **Selection ≠ quality.** The 4/5ths rule measures adverse impact, not whether criteria predict job performance. Validity checks against later performance are a separate study.
- **Derived tier list** is illustrative and must be reviewed before production use.



D5 · DELIVERABLE

Project Plan

Incremental WBS that keeps screening running, the parsing work package, estimates, PERT/CPM, Gantt, sprints and a risk register including reputational risk.

How the rebuild is delivered in seven increments without stopping screening, with a measurable acceptance threshold for parsing and a schedule computed from the activity network.

Contents

1	Project charter	2	Incremental delivery: screening never stops
3	Work breakdown structure	4	Work package WP-P: parsing quality
5	Effort and cost estimation (COCOMO)	6	Schedule: PERT and critical path
7	Agile execution: sprints, backlog, burndown	8	Risk register
9	Governance: RACI, communication, configuration management		

AUTHOR
Anshuman Atrey
Roll 150096724029

COURSE
Software Engineering & Project
Management
B.Tech CSE · Batch 2024–28 · Semester
V

DOCUMENT
D5 · v1.0
1 October 2026 · ITM Skills University

1 Project charter

Project	Rebuild of the applicant-screening system as HireSense
Sponsor	Head of Talent
Problem	Unexplainable ranking that tracks university tier; résumé parsing fails silently on 29% of documents; no rejection log; no bias audit
Objectives	Explicit, configurable, versioned criteria · every automated rejection logged with a reason · parse failures never silent · recurring bias audit with a defined consequence · screening never stops during the rebuild
Measurable success criteria	Parse success $\geq 95\%$ and silent failures $\leq 1\%$ on the evaluation corpus · 100% of decisions logged with reason and criteria version · monthly audit on time 100% · zero days without screening · all Must requirements in the SRS verified
Scope	Parsing, criteria, screening, ranking, decision log, review queue, bias audit, legacy shadow comparison
Out of scope	Interview scheduling, offers, job board, payroll
Constraints	Case-study figures used as given; 90,000 applications/year; legacy must keep running until cut-over
Team	1 project lead / scrum master · 2 engineers · 1 QA · 0.5 talent-ops SME · 0.25 audit/compliance lead

2 Incremental delivery: screening never stops

The rebuild follows the **strangler-fig** pattern. Each increment is shippable on its own and adds a capability *around* the legacy system before replacing any part of it. Legacy stays the system of record until the increment's exit criteria are met, and every role has a rollback switch back to legacy.

INCREMENT	DELIVERS	SCREENING DURING THIS INCREMENT	EXIT CRITERIA
I0 Foundations	Evaluation corpus with ground truth; append-only decision log; legacy decisions shadow-logged	Legacy only (now logged)	Legacy reproduces 71% on the corpus; 100% of legacy decisions logged
I1 Stop silent rejects	Confidence gate wrapped around the <i>legacy</i> parser	Legacy + gate: low-confidence rejects go to review	0 automated rejections at confidence < 0.6
I2 Parser v2 + OCR	Layout-aware parser, anomaly cap, OCR fallback (WP-P)	Legacy primary; v2 in shadow	$\geq 95\%$ correct, $\leq 1\%$ silent on the corpus
I3 Criteria + routine	Versioned criteria, editor, preview; specified routine; rank keys; explain view	Canary: one role on HireSense, others legacy	Canary role: 2 weeks, review queue age < 2 days, no Sev-1 defects
I4 Bias audit	Audit service, specification (D4), scheduler, breach procedure	Canary + audits on all roles (shadow)	Two scheduled audits completed; breach drill passed
I5 Review + AI assist	Review queue, verified re-screen, grounded AI extraction	Canary	Median review time recorded; no ungrounded values accepted
I6 Parallel run + cut-over	All roles on HireSense with legacy in shadow; then decommission	HireSense primary, legacy shadow, then legacy off	Two audit cycles pass; sponsor sign-off

3 Work breakdown structure

Deliverable-oriented WBS. Level-1 elements follow the increments; work packages map to the schedule activities (§6). The parsing work package **WP-P** is defined in §4.

WBS	ELEMENT / WORK PACKAGE	INCREMENT	ACTIVITY	DELIVERABLE
1	Project management	all	—	Charter, plan, status reports
1.1	Charter and baseline metrics	I0	A	Signed charter
1.2	Sprint ceremonies, risk and change control	all	—	Sprint reviews, risk register
2	Foundations	I0		
2.1	Evaluation corpus and ground truth	I0	B	Corpus (legacy reproduces 71%)
2.2	Append-only decision log and shadow logging	I0	C	decision_log, triggers
3	Parsing (WP-P)	I1–I2		
3.1	Confidence gate around the legacy parser	I1	D	No silent rejects in production
3.2	Parser v2: layout, sections, dates, confidence	I2	E	parser v2.x
3.3	OCR fallback for scanned PDFs	I2	F	ocr.ts
3.4	CI evaluation gate	I2	E/F	Blocked releases below threshold
4	Criteria and screening	I3		
4.1	Criteria model, versions, editor and preview	I3	G	criteria_versions, editor
4.2	Specified routine, rank keys, explain view	I3	H	routine.ts, order.ts
4.3	Canary on one role	I3	I	Canary report
5	Bias audit	I4	J	Audit service, D4, breach drill
6	Review and UX	I5	K	Review queue, AI assist
7	Transition	I6		
7.1	Parallel run on all roles (2 audit cycles)	I6	L	Parallel-run report
7.2	Cut-over, legacy decommission, handover	I6	M	Sign-off

4 Work package WP-P: parsing quality

The case study gives the current parsing quality as a **29% failure rate** and asks for an acceptance threshold, plus the effort to reach it, defined as a work package.

FIELD	DEFINITION
Baseline	Legacy parser: 71% correct, 29% failing; every failure is silent (no confidence, no error)
Correct parse	All decision fields right: skills ($F1 \geq 0.8$), years of experience (± 1 year) and city, against ground truth
Acceptance threshold	$\geq 95\%$ correct on the evaluation corpus, and $\leq 1\%$ silent failures (wrong but confidence ≥ 0.6), and $\geq 90\%$ correct in every layout class
Measurement	seed/evaluate.ts on every build; release blocked in CI below threshold
Owner / duration	Engineer 1 · activities E + F · 29.0 working days expected (PERT)

TASK	EFFORT (PERSON-DAYS)	OUTPUT
Evaluation corpus with layout classes and ground truth	4	2,000 résumés; 29% hard layouts
Reading-order reconstruction (two-column detection)	6	pdf.ts readingOrder
Section headings: synonyms, letter-spacing	3	parser.ts headings
Date-range grammar (7 formats + OCR variants)	3	dates.ts
Confidence model, anomaly cap, gate	3	WEIGHTS, ANOMALY_CAP
OCR fallback (rasterise + PP-OCrv6) and engine benchmark	5	ocr.ts; PP-OCrv6 vs tesseract.js
Evaluation harness + CI gate	3	evaluate.ts

TASK	EFFORT (PERSON-DAYS)	OUTPUT
Regression tests for every defect found	2	parser.test.ts
Total	29	≈ activities E + F in the schedule

BASELINE CORRECT

71.0%

legacy

ACHIEVED CORRECT

99.0%

target ≥ 95%

SILENT FAILURES

0.30%

target ≤ 1%

WORST LAYOUT CLASS

88%

target ≥ 90%

Status: 2 of 3 acceptance criteria met. Overall correctness ✓ · silent failures ✓ · every layout class ≥ 90% × (unusual dates: 88%).

The gap is season-style dates ("Summer 2023 – Spring 2025"), which the date grammar does not read. Of the 17 failures in this class, the anomaly cap routes 14 to review; 3 remain silent (a partially read date range). **Remaining task:** add a season grammar (≈ 1 person-day), then re-run the evaluation gate.

Without OCR, v2 reached 91.1% overall. The 8% of scanned résumés made the OCR task necessary for the 95% target.

5 Effort and cost estimation (COCOMO)

Basic COCOMO, organic mode (a small team, familiar domain), sized from the **actual prototype**: 4,805 non-blank, non-comment lines in the API, seed generator and web client. Generated UI-library code and tests are excluded.

QUANTITY	FORMULA	VALUE
Size	KLOC	4.80
Effort	$E = 2.4 \times \text{KLOC}^{1.05}$	12.5 person-months
Duration	$D = 2.5 \times E^{0.38}$	6.5 months
Average staff	$E \div D$	1.9 people
Cost	$E \times ₹1,25,000 / \text{person-month (blended, assumption)}$	₹15.6 lakh

Cross-check against the bottom-up plan. The PERT schedule (§6) has an expected critical path of 83 working days (≈ 4.0 months). With about 4 FTE (1 lead, 2 engineers, 1 QA, plus part-time SMEs), that is ≈ 15.8 person-months, against COCOMO's 12.5 person-months over 6.5 months. The two estimates agree within 27%. The plan is shorter than COCOMO's duration because it uses more people in parallel than COCOMO's 1.9. COCOMO is calibrated on conventional projects, so it is used here as an independent sanity check.

6 Schedule: PERT and critical path

ID	ACTIVITY	INC.	PRED.	O	M	P	T _E	ES	EF	SLACK
A	Charter, stakeholder sign-off, baseline metrics	IO	—	4	6	10	6.3	0.0	6.3	critical
B	Evaluation corpus + ground truth (legacy reproduces 29%)	IO	A	6	10	16	10.3	6.3	16.7	critical
C	Append-only decision log + shadow logging of legacy	IO	A	6	8	12	8.3	6.3	14.7	4.3

ID	ACTIVITY	INC.	PRED.	O	M	P	T _E	ES	EF	SLACK
D	Confidence gate around legacy parser (stop silent rejects)	I1	B, C	4	6	10	6.3	16.7	23.0	18.7
E	Parser v2: layout, sections, dates, confidence (WP-P)	I2	B	12	18	28	18.7	16.7	35.3	critical
F	OCR fallback for scanned PDFs (WP-P)	I2	E	6	10	16	10.3	35.3	45.7	critical
G	Criteria model, versioning, editor + preview	I3	C	10	14	20	14.3	14.7	29.0	4.3
H	Specified routine, rank keys, explain-A-vs-B	I3	G	6	8	12	8.3	29.0	37.3	4.3
I	Canary: one role on HireSense, legacy in shadow	I3	D, E, H	10	14	20	14.3	37.3	51.7	4.3
J	Bias audit service, spec, scheduler, breach procedure	I4	H	8	12	18	12.3	37.3	49.7	6.3
K	Review queue + AI-assisted extraction	I5	F, H	6	10	16	10.3	45.7	56.0	critical
L	Parallel run, all roles (2 audit cycles)	I6	I, J, K	16	20	28	20.7	56.0	76.7	critical
M	Cut-over, legacy decommission, handover	I6	L	4	6	10	6.3	76.7	83.0	critical

$t_e = (o + 4m + p) / 6$, in working days. **Critical path: A → B → E → F → K → L → M.** Expected duration **83.0 days** ($\sigma \approx 4.6$), so there is a ~95% chance of finishing within 91 working days, by 2027-02-16. The project starts on 2026-10-12.

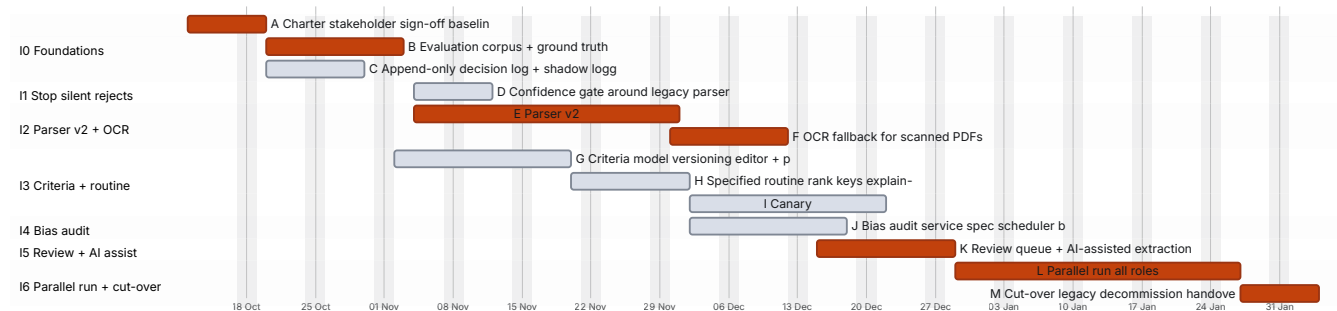


Figure 1. Gantt chart from the CPM early-start dates (working days; critical activities highlighted).

7 Agile execution: sprints, backlog, burndown

SPRINT	WEEKS	GOAL (INCREMENT)
S1	1–2	I0 foundations + I1 confidence gate
S2	3–4	I2 parser v2 (layout, sections, dates)
S3	5–6	I2 OCR + I3 criteria model and versions
S4	7–8	I3 routine, rank keys, explain; canary starts
S5	9–10	I4 bias audit + self-ID
S6	11–12	I5 review queue, AI assist, CSV export
S7	13–14	I6 parallel run all roles + CI gate
S8	15–16	I6 cut-over, decommission, handover

Product backlog

ID	USER STORY	PRIORITY	POINTS	SPRINT
US-01	As an audit lead, I want every automated rejection logged with a reason, so that any decision can be defended.	Must	5	S1
US-02	As a recruiter, I want unreadable résumés routed to a person instead of rejected, so that good candidates are not lost.	Must	5	S1
US-03	As engineering, I want an evaluation corpus with ground truth, so that parser quality is measured, not guessed.	Must	8	S1
US-04	As a recruiter, I want two-column and creative résumés parsed correctly.	Must	8	S2
US-05	As a recruiter, I want unusual date formats understood, so that experience is not undercounted.	Should	5	S2
US-06	As a reviewer, I want scanned PDFs read by OCR, so that the review queue shrinks.	Should	8	S3
US-07	As a criteria owner, I want to edit mandatory skills, bands, threshold, city and remote without a deploy.	Must	8	S3
US-08	As a criteria owner, I want every change versioned with a note, so that I can see which version decided what.	Must	3	S3
US-09	As a recruiter, I want to see why candidate A ranks above B.	Must	5	S4
US-10	As a criteria owner, I want to preview the impact and fairness of a change before publishing.	Should	5	S4
US-11	As an audit lead, I want a monthly bias audit with selection rates and impact ratios per declared group.	Must	8	S5
US-12	As an audit lead, I want a significant breach to pause automated rejection automatically.	Must	5	S5
US-13	As a candidate, I want to self-identify voluntarily, knowing screening never sees it.	Must	3	S5
US-14	As a reviewer, I want a queue sorted by age with the reason and the withheld outcome.	Must	3	S6
US-15	As a reviewer, I want AI-suggested field values that are grounded in the résumé text.	Could	5	S6
US-16	As an audit lead, I want to export the decision log as CSV.	Should	2	S6
US-17	As the Head of Talent, I want a parallel run on all roles before legacy is switched off.	Must	8	S7
US-18	As engineering, I want a CI gate that blocks a parser release below the acceptance threshold.	Must	3	S7
US-19	As the Head of Talent, I want a clean cut-over and handover.	Must	5	S8

Total **102 story points** over 8 sprints, about 13 points per sprint planned velocity. **Definition of Done:** code reviewed, unit tests and evaluation gate pass, decision log unaffected, docs updated, demo to the product owner.

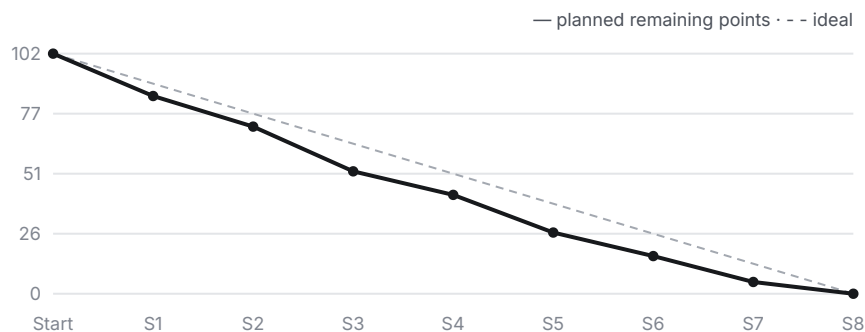


Figure 2. Release burndown (planned): remaining story points at the end of each sprint vs the ideal line. Actuals are recorded against this baseline as sprints complete.

8 Risk register

Probability (P) and impact (I) on a 1–5 scale; score = P × I. ≥ 15 high 8–14 medium < 8 low

ID	RISK	CATEGORY	P	I	SCORE	MITIGATION	CONTINGENCY	TRIGGER
R1	Reputational: a candidate or the press shows the screener discriminates (e.g. an age or tier proxy)	Reputational	3	5	15	Recurring audit; automatic pause on significant breach; documented job-relatedness; candidate-facing explanation	Pause auto-reject company-wide; public statement; external audit	Critical audit / complaint
R2	Silent parse failures continue during migration and reject qualified candidates	Quality	4	4	16	I1 ships the confidence gate in week 3, before any new parser	Route all rejects to review until gate is live	Silent-failure KPI > 1%
R3	Screening stops or slows during cut-over	Operational	2	5	10	Strangler increments; canary; legacy stays primary until exit criteria met; rollback switch per role	Flip role back to legacy	Queue age > 2 days
R4	Recruiters over-trust the new ranking (automation bias)	Human	4	3	12	Show reasons and paths, not just a rank; training; reviewers decide low-confidence cases	Sample audit of shortlists	Override rate ≈ 0
R5	Review queue overload after the gate goes live (29% of résumés)	Operational	4	3	12	OCR fallback (WP-P) cut the queue 215 → 94 in the prototype; staffing plan for reviewers	Temporary reviewers	Queue > 2 days
R6	Legal non-compliance with data-protection law for self-ID data	Legal	2	5	10	Separate table; audit-role access only; consent text; retention limits (DPDP Act 2023)	Suspend self-ID collection	Access-log anomaly
R7	Audit false alarms from small samples cause alert fatigue	Quality	3	3	9	n ≥ 30; rolling 90 days; significance test before pausing	Review thresholds quarterly	> 2 warnings/month with no cause
R8	Criteria change introduces new adverse impact	Quality	3	4	12	Mandatory preview + automatic audit on publish	Revert to previous version	Critical audit after change
R9	OCR / language model dependency unavailable or changes behaviour	Technical	2	3	6	Optional, with graceful fallback to manual review; pinned model versions; evaluation gate	Disable component	Eval drops below threshold
R10	Low self-ID response makes the audit blind	Quality	3	3	9	Explain purpose to candidates; track non-response as a KPI	Proxy-free analyses only	Non-response > 40%
R11	Decision log tampering or loss	Security	1	5	5	Append-only triggers; backups; access control	Restore from backup	Trigger violation logged
R12	Scope creep (interviews, offers)	Project	3	2	6	Scope fixed in charter; change control board	Defer to phase 2	Unplanned stories > 10%

9 Governance: RACI, communication, configuration management

ACTIVITY	PROJECT LEAD	ENGINEERS	QA	TALENT OPS	AUDIT LEAD	HEAD OF TALENT
Sprint planning & review	A/R	R	R	C	I	I
Criteria changes	I	C	I	R	C	A

ACTIVITY	PROJECT LEAD	ENGINEERS	QA	TALENT OPS	AUDIT LEAD	HEAD OF TALENT
Parser release (eval gate)	A	R	R	I	I	I
Monthly bias audit	I	C	I	C	R	A
Breach response	C	C	I	R	R	A
Cut-over decision	R	C	C	C	C	A

R responsible · A accountable · C consulted · I informed.

Configuration management and version control

- **Git** with trunk-based development: short-lived feature branches, pull requests with review, and CI running `bundle test` plus the parser evaluation gate.
- **Versioned artefacts:** the routine (code), the parser (`PARSER_VERSION` , stored on every parse), criteria (`criteria_versions` , stored on every decision), and the OCR and language models (pinned names). Any decision can therefore be reproduced exactly.
- **Change control:** criteria changes need a note and pass through preview and an automatic audit. Scope changes go to the project lead and sponsor.
- **Communication:** weekly status to the sponsor; a monthly audit report to the Head of Talent; incident notices within 1 working day of a critical audit.



D6 · DELIVERABLE

Maintenance Process

The recurring fairness audit defined as preventive maintenance: cycle, calendar, change control, KPIs and evidence.

Why a bias audit is preventive maintenance, and the recurring process that keeps HireSense fair and reliable after go-live, without depending on a one-off report.

Contents

1	Why a fairness audit is preventive maintenance	2	Sources of drift the process must catch
3	The recurring audit cycle	4	Change management for criteria and parser
5	Event-driven (unscheduled) maintenance	6	Maintenance metrics (KPIs)
7	Audit history as maintenance evidence	8	Maintenance log (from this project)

AUTHOR

Anshuman Atrey
Roll 150096724029

COURSE

Software Engineering & Project
Management
B.Tech CSE · Batch 2024-28 · Semester
V

DOCUMENT

D6 · v1.0
1 October 2026 · ITM Skills University

1 Why a fairness audit is preventive maintenance

Software maintenance is usually classified (ISO/IEC/IEEE 14764; Sommerville) as:

TYPE	PURPOSE	TRIGGER	HIRESENSE EXAMPLE
Corrective	Fix a defect after it has caused a failure	A failure is observed	BUG-04: season dates silently scored 0 years → anomaly cap added
Adaptive	Keep working in a changed environment	External change	A new OCR model or résumé tool changes text extraction
Perfective	Improve qualities or add features	User need	Intersectional audit groups; better date grammar
Preventive	Detect and remove latent faults before they become failures	Schedule, not failure	The recurring fairness audit

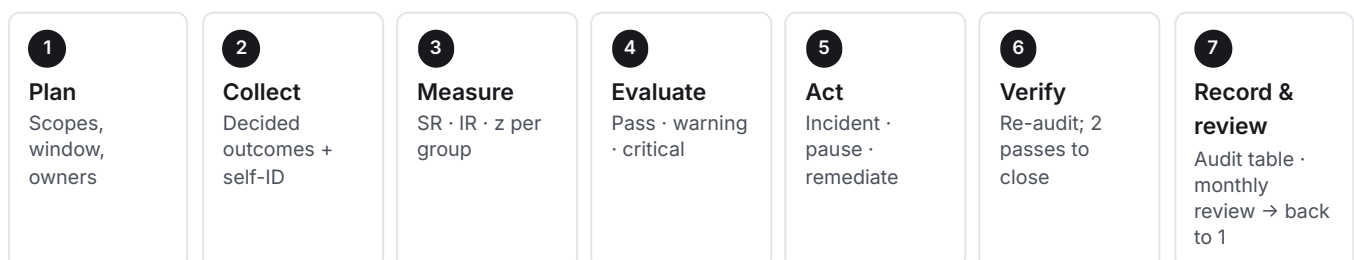
A screening system can become discriminatory **without any code change**. The applicant pool shifts, a recruiter edits a band, a new parser reads one layout better than another, or a criterion turns out to proxy a protected characteristic. These are **latent faults**: nothing crashes, and the harm accumulates silently, just like the silent parse failure. The fairness audit runs on a schedule, looks for the fault before anyone is harmed at scale, and triggers a fix. Running on a schedule to prevent failure is what makes it **preventive maintenance**.

The seeded data shows the point. HireSense never reads age, yet the audit found that the Senior Frontend role's experience bands select the 18–29 band at an impact ratio of about 0.46 (D4 §8). No test of the code could have found this, because it is a property of *criteria × applicant population*. Only recurring measurement of outcomes reveals it.

2 Sources of drift the process must catch

SOURCE	EXAMPLE	DETECTED BY
Applicant population	More early-career applicants after a campus drive	Scheduled audit (rolling 90 days)
Criteria edits	Threshold raised from 75 to 85	Preview before publish + automatic audit after
Parser / OCR release	New version reads two-column layouts differently	Evaluation gate + post-release audit
Skill taxonomy update	An alias added or removed ("JS" vs "Node.js")	Evaluation gate + audit
Proxy variables	Experience as an age proxy; location as a socio-economic proxy	Audit by every declared dimension
Process drift	Reviewers start overriding in one direction	Human-decision share and outcome KPIs

3 The recurring audit cycle



Monthly preventive-maintenance cycle (plan-do-check-act). Steps 2–4 and the containment in step 5 are automated; analysis and remediation are human.

CADENCE	ACTIVITY	OWNER	OUTPUT
Monthly (1st, 06:00)	Scheduled audit of every role + company-wide, rolling 90 days	System / audit lead	audits rows; incidents
Monthly (first week)	Governance review: open incidents, KPIs (§6), criteria changes of the month	Audit lead + Head of Talent	Minutes, actions
Weekly (while a warning is open)	Escalated audit of the affected role	System	audits rows
Quarterly	Deep dive: threshold review ($n \geq 30$, $z \geq 1.96$), proxy analysis, self-ID non-response	Audit lead	Threshold decision record
Per release	Parser/OCR/taxonomy release gated on the evaluation corpus, then audit	Engineering	Eval report, audit
Annually	Independent review of the audit process and the tier list	External reviewer	Review report

4 Change management for criteria and parser

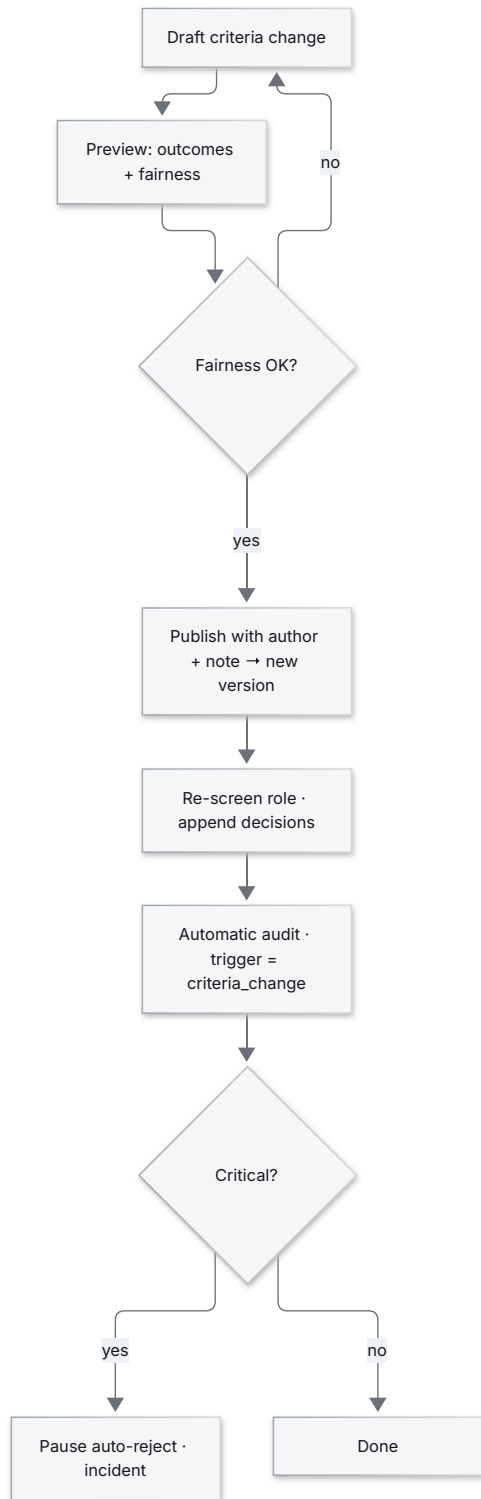


Figure 1. Criteria change workflow.

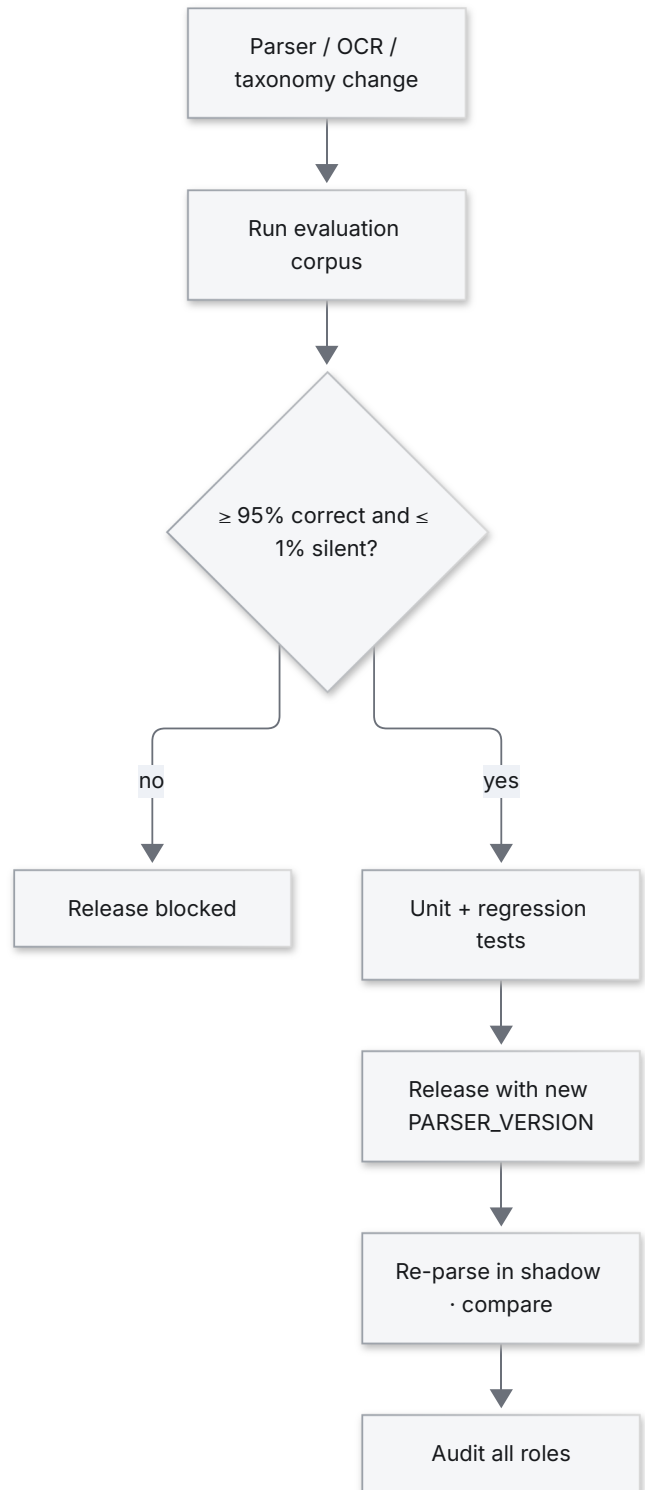


Figure 2. Parser release gate.

Every corrective fix ships with a **regression test** that would have caught the defect (D1 §8). Each release records the parser version on every parse and the criteria version on every decision, so any past decision can be explained with the exact rules and parser that produced it.

5 Event-driven (unscheduled) maintenance

EVENT	MAINTENANCE TYPE	RESPONSE
Candidate complaint or regulator query	Corrective / preventive	On-demand audit of the role; decision trace and log export within 2 working days
Critical audit	Preventive → corrective	Containment (pause) is automatic; root cause and remediation per D4 §7
New job family (e.g. non-tech roles)	Adaptive	Extend the taxonomy; evaluation corpus for the new layouts; audit after 90 days
Review queue age > 2 days	Perfective	Investigate the routing reasons; improve the parser for the dominant cause
Dependency update (OCR, model, PDF library)	Adaptive	Evaluation gate before deploy; pin versions

6 Maintenance metrics (KPIs)

SCHEDULED AUDITS 63 7 scopes × 9 months	AUDITS WITH A BREACH 27 14 critical	PARSE SUCCESS 99.0% target ≥ 95%	SILENT FAILURE RATE 0.30% target ≤ 1%
--	--	---	--

KPI	DEFINITION	TARGET	CURRENT (SEEDED DATA)
Audit timeliness	Scheduled audits completed on the 1st ÷ due	100%	100% (seeded schedule)
Mean time to remediate (MTTR) a critical breach	Critical audit → first passing re-audit	≤ 30 days	Open: Senior Frontend (age band)
Breach rate	Audits with a breach ÷ scheduled audits	Trend ↓	43%
Parse success / silent failures	Evaluation corpus, per release	≥ 95% / ≤ 1%	99.0% / 0.30%
False-alarm routing	Correct parses sent to review ÷ all parses	≤ 1%	0.65%
Review queue	Applications awaiting a person	Age ≤ 2 days	94 waiting
Self-ID non-response	'Prefer not to say' ÷ applicants	≤ 40%	4%
Criteria change audits	Changes followed by an automatic audit	100%	100% (enforced by the API)

7 Audit history as maintenance evidence

Scheduled audits per scope (rolling 90-day windows, seeded data). Reading the history the way the monthly governance review would:

- **Persistent, real disparity.** The age band (18–29) breaches in every company-wide window and repeatedly for Senior Frontend Engineer. It comes from the experience requirement (D4 §8), and the role is paused now.
- **False alarms.** Data Analyst (university tier, two windows) and Senior Frontend (gender, one window) were graded critical, although the demo data makes tier and gender *independent* of outcomes. With about 567 group comparisons over the 9 scheduled months at the 5% level, a few chance "significant" results are expected. The pause → re-audit loop limited the cost: each cleared within one or two windows.
- **Warnings that clear on their own (W)** are the non-significant breaches. They correctly did not pause any role.

Maintenance action from this review (perfective): apply a multiple-comparison correction (Holm–Bonferroni across the groups and dimensions of a scope) before grading a breach as critical, and re-evaluate at the next quarterly deep dive. This is exactly the feedback loop a preventive-maintenance process should produce: the audit audits itself.

SCOPE	2026-01	2026-02	2026-03	2026-04	2026-05	2026-06	2026-07	2026-08	2026-09
Company-wide	C	C	C	C	C	C	C	C	C
Senior Frontend Engineer	C	✓	W	W	C	W	✓	W	C
Backend Engineer (Java)	✓	✓	✓	✓	W	W	W	W	W
Data Analyst	✓	✓	✓	W	C	✓	C	W	✓
DevOps Engineer	✓	✓	✓	✓	✓	✓	✓	✓	✓
QA Automation Engineer	✓	✓	✓	✓	✓	W	✓	✓	✓
Sales Development Representative	✓	✓	✓	✓	✓	✓	✓	W	✓

✓ pass · W warning (breach, not significant) · C critical (significant breach → auto-reject paused for a role, or escalation company-wide).

8 Maintenance log (from this project)

Corrective maintenance performed while building the prototype, classified. Each item has a regression test (D1 §8).

CHANGE	TYPE	EFFECT
Letter-spaced headings collapsed before matching	Corrective	721 standard résumés fixed for both parsers
Earliest-occurrence city match	Corrective	95 wrong cities fixed
Two-column detection: balance 8% + side-by-side check	Corrective	24 résumés fixed
Anomaly cap for unreadable sections	Preventive	New class of silent failure made visible
OCR fallback (PP-OCRv6 tiny)	Perfective	Parse success 91.1% → 99.0%; review queue 215 → 94
OCR dash/space variants in dates	Adaptive	Scanned résumés 120 → 157 of 160 correct
Grounding: employer must appear in the text	Corrective	AI assist no longer invents jobs
Significance test before pausing	Perfective	Noise-driven 'critical' audits removed
Null ratio when < 2 comparable groups	Corrective	No more vacuous 1.0 ratios